

Auditar plataforma Coroado

> preciso que voce avalie a Plataforma-Coroado completamente:

>

> Preciso que voce estude minuciosamente cada pagina e cada arquivo de codigo, e tudo que temos a respeito da plataforma, e sua integraçao entre cada area, e como cada pagina quase que funciona como uma plataforma a parte dentro da plataforma Coroado. e como essa intercomunicação deve acontecer com fluidez entre cada uma dessas paginas, nao apenas comunicar mas armazenar, consumir, alterar, avaliar dados entrelaçados, então precisamos avaliar tudo que possa ser melhorado.

>

> Alem de que preciso que voce sugira evoluções, ajustes e implementação de funcionalidades que concedam mais autonomia ao uso da plataforma, mas sem criar dificuldades ou barreiras tecnicas.

>

> mapear dados que sao recorrentes e podem ser lançados em tabelas, para que futuramente virem indicadores, fluxos que precisam ser melhor desenhados, é uma auditoria robusta mesmo, como um especialista em tecnologia faria, com um time multidisciplinar, olhando partes de arquitetura, partes de backend, frontend, dados, e etc. apontando erros e inconsistencias identificados, por exemplo, em Celulas tem botões que nao funcionam e nao abrem nenhum formulario, quantas outras paginas tbm estao nesse estao, sem finalidade mapeada e implementada.

>

> Preciso que crie essa auditoria completa mesmo dessa plataforma

>

> alem do que temos na nossa pasta local @Coroado, acredito que o principal diretorio do github :

> é este: [Mphubner/Plataforma-Coroado-v2](https://github.com/Mphubner/Plataforma-Coroado-v2)

>

> mas tbm temos evoluções que vieram ou podem vir de:

> [Mphubner/Plataforma-Coroado](https://github.com/Mphubner/Plataforma-Coroado)

> [Mphubner/Igreja-Coroado](https://github.com/Mphubner/Igreja-Coroado)

<details><summary>29 previous messages</summary>

> Vou tratar isso como uma auditoria de produto e tecnologia, não só uma olhada rápida em telas. Primeiro vou levantar a estrutura local e comparar com os repositórios que você indicou, para separar o que é implementação ativa, o que é legado e o que pode ser aproveitado.

>

> Já encontrei uma memória recente útil: este projeto teve uma triagem anterior apontando que o alvo ativo é `D:\Projetos\Coroado\Plataforma-Coroado`, e que existem variantes Firebase/AI Studio e Supabase que não devem ser misturadas sem comparação. Vou usar isso como mapa inicial, mas vou revalidar no código atual antes de concluir qualquer coisa.

>

> Vou usar três lentes no relatório: GitHub/código para comparar variantes, produto/fluxos para avaliar a experiência entre páginas, e dados/KPIs para transformar campos recorrentes em indicadores futuros. Agora vou mapear a aplicação atual, dependências, páginas, serviços, regras e pontos de integração.

>

> Antes de escrever o diagnóstico final, vou deixar o levantamento bem apoiado: estrutura real do app, telas existentes, dados que cada uma toca, permissões, integrações e o que está só desenhado na interface. Também vou tentar abrir a plataforma localmente para testar os sintomas visíveis, como botões sem ação.

>

> A estrutura local está confirmada e o repositório está em `main`, mas com mudanças já existentes em vários arquivos centrais. Vou preservar essas alterações e auditar o estado atual sem reverter nada; isso é importante porque parte do que você quer avaliar provavelmente está justamente nessas evoluções recentes.

>

> O relatório antigo no projeto diz que várias sprints estão “concluídas”, mas ele próprio aparece com acentuação corrompida e precisa ser tratado como hipótese, não como verdade. Vou conferir no código se essas conclusões batem com a implementação real.

>

> Já apareceram alguns sinais fortes: há integrações reais começadas com Firestore, Functions, Mercado Pago e WhatsApp, mas também há simulações e chamadas externas incompletas misturadas com fluxo de produção. A auditoria vai separar “funciona hoje”, “parece funcionar mas depende de configuração”, e “é interface sem finalidade operacional completa”.

>

> O núcleo já evoluiu para rotas reais e permissões centralizadas, o que é um bom avanço. O ponto frágil agora é que cada página cresceu como um “mini-sistema” com seus próprios dados, atalhos e regras, então vou mapear onde elas conversam entre si e onde ainda estão isoladas.

>

> Vou automatizar parte do inventário para não depender só de leitura manual: listar coleções do Firestore usadas por página, integrações externas, botões com aparência de ação, estados simulados e textos de “em breve/simulado”. Depois eu volto aos arquivos grandes para validar os achados.

>

> A verificação automática confirmou vários botões sem ação direta: Escola, Células, Metas, Cuidado Pastoral, Social e Loja têm comandos visíveis que parecem acionáveis, mas em alguns casos não chamam formulário, modal, backend ou navegação. Esse será um bloco próprio da auditoria, porque afeta confiança do usuário na plataforma.

>

> O repositório local já aponta para `Plataforma-Coroadov2`, que também é o mais recente no GitHub. Ainda vou comparar as outras duas bases porque elas podem ter modelagens úteis, mas a auditoria principal deve tomar a v2/local como fonte de verdade.

>

> A comparação remota começou: `Plataforma-Coroadov2` é muito parecida em estrutura com a v2, enquanto `Igreja-Coroadov2` é outra arquitetura, com Supabase, `DataContext` e páginas mais separadas. Isso deve entrar como

recomendação: aproveitar a modelagem da variante Supabase como referência, não trocar a base ativa por ela.

>

> O clone antigo de `Plataforma-Coroadó` tem a mesma família de componentes, mas a v2/local ganhou backend/Functions e integrações novas. Já o `Igreja-Coroadó` tem um desenho mais enxuto e uma camada de dados centralizada, útil como inspiração para reduzir duplicação no Firebase.

>

> Agora vou aprofundar nos pontos críticos: Células, Escola, Loja/Eventos/Financeiro, Cuidado Pastoral, Membros/Auth e regras de segurança. São os módulos onde o código mais toca dados sensíveis ou tem maior risco de “parecer pronto” sem estar operacionalmente fechado.

>

> Context compaction

>

> Source: automatic

> Status: completed

>

> Vou fechar a parte de validação agora: conferir regras x coleções usadas, rodar as verificações do projeto e transformar os achados num documento de auditoria bem utilizável, não só numa lista solta de problemas.

>

> A checagem base passou: TypeScript e build do frontend estão verdes. Há um alerta importante de pacote grande no build, então vou registrar isso como risco de desempenho e agora vou validar a camada de funções/integrações e consolidar o mapa de coleções sem regras.

>

> A camada principal compila, mas o build das Functions falhou tentando gravar os arquivos gerados em `functions/lib`. Vou verificar se isso é bloqueio de arquivo/atributo antes de tratar como erro real de código.

>

> Os arquivos gerados não estão marcados como somente leitura. Vou repetir a compilação das Functions isoladamente; se falhar de novo, registro como problema de ambiente/build a investigar antes de deploy.

>

> A falha das Functions se repetiu no mesmo ponto de escrita, então parece mais um bloqueio local sobre artefatos gerados do que erro de TypeScript. Vou compilar para uma pasta temporária para separar “código inválido” de “saída bloqueada”.

>

> A compilação das Functions passou quando a saída foi desviada para uma pasta temporária; o problema é mesmo a escrita nos artefatos versionados em `functions/lib`. Agora vou fazer uma varredura estática dos botões sem ação clara para quantificar melhor o risco que você apontou em Células.

>

> A varredura achou um padrão bem claro: não é só Células. Há dezenas de botões visualmente “ativos” sem ação própria, concentrados em Escola, Células, Ministérios, Loja e Pastoral. Vou cruzar isso com simulações, pagamentos e permissões para priorizar por impacto real.

>

> O cruzamento confirmou um problema de “contrato” entre tela e banco: algumas telas já tentam gravar dados úteis, mas as regras esperam outro formato, outro papel ou outra coleção. Esse tipo de inconsistência é o que faz a plataforma parecer pronta visualmente e falhar justamente na hora de operar.

>

> Vou criar a auditoria como um arquivo Markdown dentro da própria pasta da plataforma, com priorização, mapa de módulos, contratos de dados e um plano de entregas. Assim ela vira uma peça de trabalho para guiar as próximas implementações, não só um diagnóstico solto.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroador/AUDITORIA\_PLATAFORMA\_COROADO\_2026-06-14.md` (+1131 -0)

>

> </details>

>

> A auditoria foi materializada em Markdown dentro da pasta da plataforma. Vou só conferir o arquivo gerado e pegar as referências de memória corretas para fechar com rastreabilidade.

>

> O relatório está com cerca de 40 KB, então ficou num nível de detalhe adequado para servir como mapa de execução. Vou finalizar checando o status para deixar claro que só adicionei a auditoria e não mexi no código da plataforma.

>

> Vou acrescentar uma nota curta de limites da auditoria no próprio documento, para diferenciar o que foi confirmado por código/build do que ainda precisa de teste com sessão real e dados de produção.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/AUDITORIA\_PLATAFORMA\_COROADO\_2026-06-14.md` (+15 -1)

>

> </details>

</details>

Criei a auditoria completa aqui: [AUDITORIA_PLATAFORMA_COROADO_2026-06-14.md](./Plataforma-Coroado/AUDITORIA_PLATAFORMA_COROADO_2026-06-14.md).

Ela cobre arquitetura, frontend, backend, Firestore, integrações, dados recorrentes, KPIs, fluxos por página, inconsistências, botões sem ação, pagamentos simulados, regras ausentes e um plano por entregas. Os principais riscos mapeados foram: coleções usadas sem regras (`orders`, `campaigns`, `pastors`, `units`, etc.), contratos de dados incompatíveis com as regras, simulações perigosas em pagamentos/assinaturas, integrações Google expostas

no frontend, dashboards misturando dados reais e fixos, e cerca de 70 botões/ações sem finalidade clara na varredura estática.

Validei também o projeto: `npm run lint` passou, `npm run build` passou, e as Functions compilam quando a saída vai para uma pasta temporária. O build padrão das Functions falha ao tentar sobrescrever `functions/lib/index.js`, então registrei isso como ponto técnico antes de deploy. Não alterei o código da plataforma; só adicionei o documento da auditoria. As modificações que já existiam em outros arquivos foram preservadas.

> Faça todas as implementacoes para solucionar todos os pontos mapeados na Auditoria. Deixo aqui meu questionamento se faz sentido migrar do firebase (noSQL) para a solucao SQL so google tbm?

<details><summary>107 previous messages</summary>

> Vou tratar isso como uma implementação por entregas, começando pelos pontos que impedem operação real: regras/contratos de dados, simulações perigosas e fluxo de pagamento. Sobre SQL: minha recomendação inicial é não fazer uma migração total agora; faz mais sentido estabilizar o modelo atual e, se necessário, evoluir para um híbrido com Cloud SQL/Postgres para relatórios, financeiro e relações complexas.

>

> Primeiro vou ler o estado atual dos arquivos tocados e das regras para não sobrescrever alterações já existentes. Depois começo pelos reparos que desbloqueiam páginas inteiras: regras para coleções novas, alinhamento de schemas e bloqueio de simulações em produção.

>

> O tamanho é grande demais para uma mudança única e cega, então vou começar pela fundação que destrava tudo: contratos de dados e segurança. Essa etapa já responde a vários sintomas visíveis, porque hoje algumas páginas falham não por UI, mas porque o banco bloqueia ou espera outro formato.

>

> Também vou responder ao ponto estratégico enquanto implemento: SQL ajuda muito para relatórios e relações fortes, mas trocar a fundação agora atrasaria a correção dos bugs já mapeados. O caminho mais pragmático é preparar os contratos como se fossem relacionais, manter Firestore para operação em tempo real e deixar a porta aberta para Cloud SQL/BigQuery depois.

>

> <details><summary>Searched the web</summary>

>

> <details><summary>Searched the web</summary>

>

> - Searched the web for `Google Cloud Firestore documentation NoSQL scalable realtime mobile web apps official`

>

> </details>

>

> </details>

>

> Encontrei outro detalhe útil: o projeto ainda não tem um `manualChunks` ativo no Vite desta cópia, então o alerta de bundle grande é explicável. Vou incluir isso na leva depois de fechar segurança/dados.

>

> Vou mudar alguns fluxos para “falhar fechado”: se o gateway ou backend não estiver pronto, a tela avisa e não registra pagamento aprovado. Isso é menos vistoso, mas é o passo certo para a plataforma deixar de ser demo e virar operação confiável.

>

> Vou começar editando as regras do Firestore. A ideia é destravar as coleções que já são usadas pelas telas e, ao mesmo tempo, impedir que o cliente grave pagamentos como concluídos.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/firestore.rules` (+109 -8)

> - Wrote `./Plataforma-Coroado/firestore.rules` (+20 -13)

> - Wrote `./Plataforma-Coroado/firestore.rules` (+144 -23)

>

> </details>

>

> As regras agora cobrem as coleções críticas e bloqueiam pagamento concluído gravado direto pelo cliente. Vou passar para o backend: lead público via API, checkout calculando preço no servidor e webhook atualizando pedido, que é a espinha dorsal da loja real.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/server.ts` (+226 -73)

>

> </details>

>

> Vou conferir o backend imediatamente porque mexi numa área sensível de webhook. Se aparecer erro de sintaxe, corrijo antes de avançar para as telas.

>

> Achei a área do webhook e vou corrigir um fechamento de bloco antes de continuar. Essa é exatamente a razão de validar cada camada pequena antes de empilhar novas mudanças.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/server.ts` (+1 -1)

>

> </details>

>

> Agora vou trocar o formulário “Novo Aqui?” para passar pelo backend público, em vez de tentar escrever direto no Firestore como visitante anônimo. Isso resolve um dos conflitos mais importantes da auditoria.

>

> A aplicação tem alguns textos com encoding quebrado, então o patch textual falhou por diferença exata. Vou editar pelo trecho estrutural menor, sem depender desses textos corrompidos.

>

> Agora ficou claro. Vou remover a escrita direta e chamar a nova API, mantendo a experiência do visitante igual por fora.

>

> Vou aplicar em duas partes menores para evitar a linha com acentuação quebrada.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/HomeView.tsx` (+1 -1)

> - Wrote `./Plataforma-Coroado/src/components/HomeView.tsx` (+15 -8)

>

> </details>

>

> Na loja, vou tirar o fallback que transformava erro de checkout em “pagamento aprovado”. Se o Mercado Pago/backend falhar, o pedido não será criado como pago; a pessoa recebe uma mensagem clara para tentar depois.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/StoreView.tsx` (+17 -61)

> - Wrote `./Plataforma-Coroado/src/components/StoreView.tsx` (+2 -41)

>

> </details>

>

> Vou aproveitar e fechar a seleção de tamanho/cor da loja, porque hoje esses botões parecem seleção real mas não alimentam o carrinho.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/components/StoreView.tsx` (+41 -32)

>

> </details>

>

> A loja ainda tinha delegação via Google Tasks direto pelo token salvo no usuário. Vou trocar essa ação por uma tarefa interna da própria plataforma; depois a integração Google pode ser uma automação backend, sem expor token no frontend.

>

> Vou trocar esse bloco diretamente pelo mesmo botão criando uma tarefa interna.

>

> A linha com o texto do alerta está atrapalhando o patch por encoding. Vou refatorar por pontos ASCII primeiro e depois simplifico o bloco.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/components/StoreView.tsx` (+1 -1)

>

> </details>

>

> Vou usar uma substituição mecânica para esse bloco específico, porque o texto com mojibake não casa com o patch linha a linha. Depois valido com TypeScript.

>

> A busca começou depois do rótulo do botão, então não encontrou o início. Não houve alteração por esse script. Vou localizar pelo URL do Google Tasks, que é ASCII e está dentro do bloco.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/StoreView.tsx` (+1 -1)

> - Wrote `./Plataforma-Coroado/src/components/StoreView.tsx` (+14 -20)

>

> </details>

>

> Agora vou ajustar o financeiro para não registrar contribuição como concluída sem webhook/conciliação. A tela ainda pode registrar uma intenção/pendência, mas não um pagamento aprovado.

>

> Vou aplicar os ajustes no Financeiro por nomes e status, sem depender dos textos com acentuação.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/FinanceView.tsx` (+7 -7)

>

> </details>

>

> O status já deixou de ser “completed”. Vou ajustar a mensagem visível para não falar mais em simulação.

>

> O arquivo em UTF-8 mostra o texto correto internamente, então agora o patch casa melhor.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/FinanceView.tsx` (+2 -2)

>

> </details>

>

> Também vou trocar o histórico para mostrar “Pendente” quando ainda não houve conciliação.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/FinanceView.tsx` (+3 -1)

>

> </details>

>

> Vou alinhar o relatório de célula com o contrato das regras: além dos detalhes ricos, ele precisa gravar campos indicadores (`present` , `visitors` , `summary` , `updatedAt`) para virar KPI e passar na validação.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+3 -1)

>

> </details>

>

> Vou ajustar eventos para a inscrição paga abrir o checkout real retornado pela Function e para qualquer atualização incluir `updatedAt` , alinhando com as regras.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+9 -2)

>

> </details>

>

> Também vou remover o botão de “simular aprovação” de eventos; aprovação de pagamento deve vir do webhook/admin, não do cliente.

>

> Mesmo problema de encoding no texto do botão; vou localizar pelo trecho ASCII `paymentStatus: 'approved'`.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+0 -13)

>

> </details>

>

> Vou endurecer as Cloud Functions do Mercado Pago: sem token fake, retorno de sandbox quando existir, e claims de aprovação mais consistentes.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/functions/src/index.ts` (+23 -13)

>

> </details>

>

> Vou ajustar a rotina de claims para preservar claims existentes e usar o estado do usuário-alvo, não do usuário que está aprovando.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/functions/src/index.ts` (+14 -4)

>

> </details>

>

> Vou tirar a assinatura simulada da Escola: se a Function não conseguir criar o link de pagamento, a tela deve avisar e manter o acesso como está.

>

> Além da assinatura, a compra avulsa de aula também estava simulada. Vou trocar por uma mensagem operacional segura até esse fluxo entrar no mesmo módulo real de pagamentos.

>

> Vou aplicar por trecho porque o bloco da compra avulsa está mais abaixo e pode ter variação de espaços.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+1 -3)

>

> </details>

>

> Vou tirar o temporizador que aprovava a aula automaticamente.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+5 -10)

>

> </details>

>

> Agora vou alinhar Pastores e Social com as novas regras: agendamentos precisam de `updatedAt`, e aprovação não deve chamar Google Calendar direto com token do usuário.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/PastorsView.tsx` (+11 -34)

>

> </details>

>

> No Social, vou corrigir o tenant/userId dos agendamentos e tirar a chamada direta ao Google Calendar.

>

> Vou fazer essas mudanças em blocos pequenos no Social para preservar o restante da tela.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SocialView.tsx` (+3 -2)

> - Wrote `./Plataforma-Coroado/src/components/SocialView.tsx` (+7 -3)

>

> </details>

>

> Agora vou substituir a aprovação com Calendar direto por atualização interna do agendamento.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SocialView.tsx` (+4 -30)

>

> </details>

>

> Vou garantir que o Portal Social receba `userData`; sem isso ele gravava agendamentos em tenant padrão e usuário anônimo.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SocialView.tsx` (+3 -3)

>

> </details>

>

> Vou deixar a regra de pedidos compatível com pedidos antigos que talvez não tenham `paymentStatus`, para não travar a operação ao atualizar status de entrega.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/firestore.rules` (+1 -2)

>

> </details>

>

> Vou corrigir a chave do Google Maps em Membros para o padrão correto do Vite; isso evita depender de `process.env` no navegador.

>

> Essa é uma correção pequena, mas importante para ambiente real.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/MembersView.tsx` (+4 -4)

>

> </details>

>

> Vou remover a persistência de token Google no perfil do usuário. A conexão pode continuar indicando estado visual, mas o token não deve ficar gravado como dado comum no Firestore.

>

> Vou trocar a gravação do token por um carimbo de conexão, sem armazenar o segredo.

>

> Vou manter o fluxo de popup por enquanto, mas só salvar metadados seguros.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/Layout.tsx` (+12 -18)

>

> </details>

>

> Vou limpar o import que ficou sobrando depois de remover o uso do token.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/Layout.tsx` (+1 -1)

>

> </details>

>

> Vou resolver o alerta de bundle grande com divisão de pacotes por famílias de dependência, já que a plataforma tem módulos bem independentes.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/vite.config.ts` (+20 -5)

>

> </details>

>

> Vou permitir no contrato do usuário apenas os metadados seguros de conexão Workspace, sem token.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/firestore.rules` (+1 -1)

>

> </details>

>

> E vou fazer o AuthView gravar `updatedAt` junto, porque as regras exigem esse carimbo em atualizações do próprio usuário.

>

> O import já está disponível, então é só ajustar o payload.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/AuthView.tsx` (+8 -4)

>

> </details>

>

> Vou evitar que a loja em produção venda produtos mockados. Se não houver produto real cadastrado, ela mostra catálogo vazio; em desenvolvimento ainda pode servir como referência visual.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/StoreView.tsx` (+1 -1)

>

> </details>

>

> Vou checar os tipos antes de rodar tudo; Social e Store usam `userData` em pontos sensíveis.

>

> Enquanto reviso contratos, encontrei outro ponto: responder escala atualizava `assignments` sem `updatedAt`, o que as regras bloqueiam. Vou ajustar.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/HomeView.tsx` (+5 -2)

>

> </details>

>

> Vou procurar mais atualizações Firestore que mudam status sem carimbo, porque isso costuma ser a diferença entre UI bonita e regra funcionando.

>

> Em Ministérios, os briefings e aceite de escala também precisam de `updatedAt`, e a regra precisa aceitar o motivo de recusa que a UI já grava.

>

> Vou ajustar esses três updates e a regra correspondente.

>

> <details><summary>Edited 2 files, searched the web</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/MinistriesView.tsx` (+4 -4)

>

> - Wrote `./Plataforma-Coroado/firestore.rules` (+1 -1)

>

> <details><summary>Searched the web</summary>

>

> - Searched the web for `site:cloud.google.com/firestore/docs Firestore NoSQL document database real-time sync official Google Cloud`

>

> </details>

>

> </details>

>

> Vou rodar TypeScript agora. Espero alguns ajustes porque mexemos em backend, rules-adjacent e telas grandes; prefiro descobrir cedo e corrigir antes de continuar para as melhorias de UX.

>

> A primeira validação pegou um erro simples: o botão de assinar dentro do player da aula está em outro componente e não tem acesso ao modal da página principal. Vou trocar por uma ação local segura.

>

> Vou passar a ação de assinatura como propriedade para o player, em vez de tentar acessar estado de outro componente.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+3 -3)

>

> </details>

>

> Vou rodar a validação de TypeScript de novo para pegar a próxima camada de ajustes.

>

> TypeScript passou. Agora vou validar build e depois reviso se ainda existem coleções usadas sem regras.

>

> O build ficou bem menor por chunk, mas avisou sobre ciclos entre chunks. Vou simplificar a divisão para evitar esse alerta sem voltar ao pacote único gigante.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/vite.config.ts` (+1 -5)

>

> </details>

>

> Agora vou atacar os botões sem finalidade começando por Células, porque foi o exemplo que você citou e é uma área operacional central. A meta aqui é que cada botão faça algo concreto ou deixe claro que depende de dado/seleção.

>

> Vou conectar os botões principais de Células às abas/ações reais e transformar botões sem dado suficiente em feedback claro, para não parecer que quebraram.

>

> Vou editar Células por atributos/classes, evitando casar com o texto corrompido.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/components/CellManagementDashboard.tsx` (+1 -1)

>

> </details>

>

> Agora sim, vou ligar os três atalhos principais às abas certas.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+3 -3)

>

> </details>

>

> Vou passar esses callbacks a partir da aba principal.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+8 -1)

>

> </details>

</details>

> Continue

<details><summary>57 previous messages</summary>

> Continuo. Vou completar essa leva focando no que ainda ficou “meio solto”: ações de Células, ajustes finais de regras/contratos e uma rodada nova de validação para garantir que nada quebre enquanto avançamos.

>

> Vou transformar os botões restantes de Células em ações concretas de navegação/cópia/registro, e onde falta dado real vou deixar uma ação honesta que guia o operador para o fluxo correto.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+3 -2)

>

> </details>

>

> Vou adicionar QR/cópia reais no convite da célula.

>

> Vou aplicar em pedaços menores por causa dos textos acentuados antigos no arquivo.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+4 -0)

>

> </details>

>

> Agora adiciono o modal do QR antes do conteúdo da aba.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+16 -0)

>

> </details>

>

> Vou ligar os três botões do convite às ações.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/components/CellManagementDashboard.tsx` (+6 -3)

>

> </details>

>

> Vou finalizar os botões restantes da aba de membros, visitantes e escalas.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/components/CellManagementDashboard.tsx` (+7 -7)

>

> </details>

>

> Vou validar agora depois das mudanças em Células. Depois parto para ajustar algumas regras que ainda podem bloquear updates por campos extras/antigos.

>

> Células quase zerou os botões soltos; restaram dois casos. Vou resolver agora.

>

> Context compaction

>

> Source: automatic

> Status: completed

>

> Vou continuar pela linha mais prática: fechar os pontos que ainda ficaram com ação incompleta, rodar validações de novo e deixar uma resposta clara sobre a migração SQL com base no estado real da plataforma.

>

> Achei alguns resquícios pequenos: um botão de detalhes ainda sem ação em Células, um fallback de tenant que poderia falhar nas regras, e a conexão do Workspace ainda sem marca de atualização. Vou corrigir esses detalhes antes da nova rodada de testes.

>

> Vou fazer uma edição pequena e direcionada agora: deixar esses botões úteis, garantir `updatedAt` nos metadados de integração e alinhar IDs/tenant com as regras novas.

>

> A primeira tentativa bateu em diferença de acentuação/encoding no arquivo de Células. Vou aplicar as correções em blocos menores para não encostar mais do que o necessário.

>

> <details><summary>Edited 3 files</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/Layout.tsx` (+5 -3)

> - Wrote `./Plataforma-Coroado/src/components/PastorsView.tsx` (+1 -1)

> - Wrote `./Plataforma-Coroado/src/components/StoreView.tsx` (+1 -1)

>

> </details>

>

> Agora vou aproveitar os trechos estáveis do arquivo de Células, usando os valores fixos como âncoras. A ideia é trocar números estáticos por indicadores calculados sem mexer no restante visual.

>

> Esse arquivo está com alguns textos em UTF-8 sendo exibidos de forma diferente pelo terminal, então vou dividir ainda mais: primeiro cálculos e números, depois textos que dependem de acento.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+18 -4)

>

> </details>

>

> Os indicadores da célula já deixaram de ser fixos. Faltava só transformar o “Ver Detalhes” em uma ação real.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+7 -1)

>

> </details>

>

> Também encontrei que o relatório da célula ainda gravava o criador pelo perfil local. Vou alinhar isso ao usuário autenticado para casar melhor com as regras de segurança.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+5 -5)

>

> </details>

>

> Com esses ajustes, vou fazer uma varredura rápida por termos de risco e depois partir para lint/build.

>

> A validação de tipos passou. A varredura sobrou só um indicador estático no painel administrativo, então vou olhar esse bloco para decidir se dá para converter sem quebrar a tela.

>

> O painel administrativo ainda afirmava uma sincronização Cloud SQL ativa sem implementação real. Vou corrigir isso porque é exatamente o tipo de inconsistência que pode induzir decisão errada.

>

> De novo, a acentuação do arquivo exigiu cirurgia em blocos. Vou aplicar primeiro os cálculos e depois trocar o card do SQL por textos estáveis.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/AdminDashboardMetrics.tsx`
(+23 -7)

> - Wrote `./Plataforma-Coroado/src/components/AdminDashboardMetrics.tsx`
(+1 -5)

> - Wrote `./Plataforma-Coroado/src/components/AdminDashboardMetrics.tsx`
(+3 -3)

>

> </details>

>

> Pronto, removi a falsa indicação de Cloud SQL ativo e converti a saúde do painel para métricas calculadas com os dados já lançados. Vou rodar a validação novamente.

>

> Tipos passaram de novo. Vou fazer a build completa agora para pegar problemas de empacotamento ou import quebrado.

>

> A build de produção passou. Ainda há chunks grandes, mas sem erro; vou validar as funções/integrações de backend também, porque mexemos em pagamento e permissões.

>

> Funções compilaram sem erro. Vou tentar validar as regras de segurança de forma local; se a ferramenta pedir projeto/login, eu registro isso como limitação em vez de mascarar.

>

> A validação oficial das regras não conseguiu nem concluir a chamada da ferramenta local e o projeto não tem `firebase.json`. Vou seguir com checagens estáticas e deixar essa pendência explícita.

>

> A checagem estática mostra que todas as coleções usadas pelo frontend agora têm bloco de regra mapeado. Também notei que há arquivos já modificados no repositório que não são todos desta rodada; vou tomar cuidado para separar o que eu realmente mexi no resumo.

>

> O relatório ficou salvo com extensão duplicada (` .md.md `). Vou corrigir o nome do arquivo para ficar limpo e fácil de encontrar.

>

> <details><summary>Searched the web</summary>

>

> <details><summary>Searched the web</summary>

>

> - Searched the web for `site:firebase.google.com Firebase Data Connect Cloud SQL PostgreSQL official`

>

> </details>

>

> </details>

>

> Além das validações por build, vou abrir a aplicação localmente para uma checagem visual rápida das telas alteradas. Para isso vou usar o navegador interno do Codex.

>

> Vou subir a aplicação localmente em segundo plano e abrir apenas para inspeção, sem enviar dados reais nem acionar compras.

>

> Servidor iniciado. Vou confirmar que ele respondeu antes de abrir no navegador.

>

> A primeira tentativa não ficou ouvindo na porta. Vou iniciar de novo com registro em arquivo para enxergar qualquer falha sem travar a conversa.

>

> Agora vou aguardar alguns segundos e conferir se a aplicação subiu ou se o log apontou algo de ambiente.

>

> A porta 3000 já ficou ativa, embora a segunda tentativa tenha encontrado a porta ocupada. Vou usar o servidor que está respondendo e seguir para a checagem visual.

>

> MCP tool call

>

> node_repl.js

>

> ```json

> {

> "code": "const { setupBrowserRuntime } = await
import('C:/Users/marco/.codex/plugins/cache/openai-
bundled/browser/26.609.30741/scripts/browser-client.mjs');\nawait
setupBrowserRuntime({ globals: globalThis });\nglobalThis.browser = await
agent.browsers.get('iab');\nnodeRepl.write(await browser.documentation());",

```
> "timeout_ms": 30000,
> "title": "Conectar navegador interno"
>}
> ` ` `
>
> ` ` ` text
> failed to write kernel assets: O sistema não pode encontrar o caminho
especificado. (os error 3)
> ` ` `
>
> A conexão com o navegador interno falhou por problema de caminho do próprio
runtime. Vou tentar a orientação de recuperação do plugin; se não resolver, faço a
checagem com uma rota local simples e registro a limitação.
>
> MCP tool call
>
> node_repl.js
>
> ` ` ` json
> {
>   "code": "const { setupBrowserRuntime } = await
import('C:/Users/marco/.codex/plugins/cache/openai-
bundled/browser/26.609.30741/scripts/browser-client.mjs');\ntry {\n  await
setupBrowserRuntime({ globals: globalThis });\n  globalThis.browser = await
agent.browsers.get('iab');\n  nodeRepl.write(await browser.documentation());\n}
catch (error) {\n  nodeRepl.write(String(error && error.stack ? error.stack :
error));\n}",
>   "timeout_ms": 30000,
>   "title": "Tentar reconectar navegador"
>}
> ` ` `
```

>

> ```text

> failed to write kernel assets: O sistema não pode encontrar o caminho especificado. (os error 3)

> ```

>

> O navegador interno continua bloqueado pelo erro de ambiente. Vou fazer uma verificação local por HTTP e, se o Playwright estiver disponível no ambiente, uso ele como fallback para abrir a tela sem depender do plugin.

>

> A aplicação responde HTTP 200 em `localhost:3000`. Vou testar se há automação de navegador disponível para pelo menos validar renderização e navegação básica.

>

> MCP tool call

>

> node_repl.js

>

> ```json

> {

> "code": "try{\n const mod = await import('playwright');\n nodeRepl.write('playwright-ok');\n} catch (error) {\n nodeRepl.write(`playwright-missing: \${error.message}`);\n}",

> "timeout_ms": 30000,

> "title": "Verificar fallback visual"

> }

> ```

>

> ```text

> failed to write kernel assets: O sistema não pode encontrar o caminho especificado. (os error 3)

> ```

>

> O kernel usado para automação ficou preso no mesmo erro do navegador interno, então não vou insistir nele. A checagem local por HTTP está válida; a parte visual fica como pendência do ambiente.

>

> <details><summary>Called 2 tools</summary>

>

> MCP tool call

>

> node_repl.js_reset

>

> ```json

> {}

> ```

>

> ```text

> js kernel reset

> ```

>

> MCP tool call

>

> node_repl.js

>

> ```json

> {

> "code": "nodeRepl.write('ok');",

> "timeout_ms": 30000,

> "title": "Checar kernel"

```
> }  
> ```  
>  
> ```text  
> failed to write kernel assets: O sistema não pode encontrar o caminho  
especificado. (os error 3)  
> ```  
>  
> </details>  
>  
> Vou deixar também um arquivo de entrega técnica com o que foi implementado,  
validações e a recomendação Firebase/SQL, para a auditoria virar roteiro de  
continuidade e não só conversa.  
>  
> <details><summary>Created a file</summary>  
>  
> - Wrote `./Plataforma-  
Coroador/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md` (+74 -0)  
>  
> </details>  
>  
> A primeira rodada robusta está fechada: documentação, regras, pagamentos,  
botões críticos e validações principais passaram. Vou só recolher o status final  
para te entregar um resumo honesto do que mudou e do que ficou como próximo  
bloco.  
</details>
```

Concluí a primeira rodada robusta de implementação da auditoria.

Criei/ajusteí estes entregáveis principais:

- Auditoria completa: [AUDITORIA_PLATAFORMA_COROADO_2026-06-14.md](<./Plataforma-Coroado/AUDITORIA_PLATAFORMA_COROADO_2026-06-14.md>)

- Registro das implementações:
[IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md](<./Plataforma-Coroado/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md>)

O que foi implementado agora:

- Regras do Firestore ampliadas para todas as coleções usadas pelo frontend.
- Checkout da loja movido para backend, com pedido real e webhook Mercado Pago.
- Remoção de fluxos perigosos de “pagamento simulado” em loja, eventos, escola e finanças.
- Funções Firebase sem token fake do Mercado Pago.
- Remoção do armazenamento de `googleAccessToken`.
- Botões mortos de Células convertidos em ações reais ou fallback claro.
- Relatórios de célula com resumo, visitantes, presença e indicadores calculados.
- Painel admin sem falsa indicação de Cloud SQL ativo.
- Indicadores administrativos e de célula deixando de depender de números fixos.
- Build dividido em chunks maiores por domínio.

Validações feitas:

- `npm run lint` : passou.
- `npm run build` : passou.
- TypeScript das Cloud Functions: passou.
- Checagem estática Firestore: todas as coleções usadas têm regra mapeada.

- App respondeu HTTP 200 em `http://localhost:3000/`, e deixei o servidor local rodando nessa porta.

Sobre Firebase vs SQL Google: minha recomendação é ****não migrar tudo agora****. Faz mais sentido seguir híbrido. Firestore continua bom para experiência operacional em tempo real, perfis, células, presença, inscrições e telas colaborativas. Para financeiro, pedidos, pagamentos, conciliação, auditoria e BI, faz sentido evoluir para PostgreSQL usando [Firebase SQL Connect](https://firebase.google.com/docs/sql-connect) ou [Cloud SQL](https://cloud.google.com/sql). A própria documentação do [Cloud Firestore](https://firebase.google.com/docs/firestore) reforça o papel dele como banco NoSQL de sincronização, enquanto o SQL Connect é justamente a ponte Firebase com PostgreSQL gerenciado.

Limitação honesta: a validação oficial das regras via Firebase CLI não rodou porque o projeto local não tem `firebase.json` e a chamada da ferramenta travou. A checagem visual pelo navegador interno do Codex também falhou por erro de ambiente do plugin, mas a aplicação subiu e respondeu localmente.

> Crie o arquivo do firebase.json corretamente mas tenho a impressão de que ele existia sim.

>

> Mas enquanto válida isso pode avançar com os demais pontos

<details><summary>46 previous messages</summary>

> Vou primeiro localizar se o `firebase.json` existia em algum lugar do workspace ou no histórico, para não recriar uma configuração diferente da intenção original. Em paralelo, sigo olhando os próximos pontos de implementação que ainda ficaram para trás.

>

> Não encontrei `firebase.json` em `D:\Projetos\Coroador`, nem no histórico Git local desse checkout. Vou criar a configuração padrão para este projeto, amarrando `firestore.rules`, `functions/`, hosting do `dist` e emuladores.

>

> Além do `firebase.json`, vou incluir `firestore.indexes.json` vazio. Assim o deploy/validação não quebra por referência a um arquivo de índices inexistente.

>

> <details><summary>Created 2 files</summary>

>

> - Wrote `./Plataforma-Coroado/firebase.json` (+55 -0)

> - Wrote `./Plataforma-Coroado/firestore.indexes.json` (+4 -0)

>

> </details>

>

> Configuração criada. Vou validar o JSON e tentar rodar a validação do Firestore com emulador/projeto demo. Enquanto isso, vou procurar pontos restantes com maior risco: dados falsos, botões sem ação e regras sem contrato.

>

> O Firebase CLI está disponível agora. Vou usar projeto demo para compilar as regras no emulador, sem tocar no projeto real.

>

> A validação por emulador bateu em um bloqueio do ambiente: Java não está instalado no PATH. Vou registrar isso e seguir com validações estáticas, além de avançar nos pontos funcionais restantes.

>

> Vou usar o plugin local de lint de regras do Firebase, que não depende de Java, para pegar erro de sintaxe antes do emulador.

>

> O lint de regras mostrou que o `eslint.config.js` estava incompleto: ele ativava as regras, mas não informava o parser próprio de `rules`. Vou corrigir isso também, porque ajuda a validar Firestore daqui para frente.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/eslint.config.js` (+1 -9)

>

> </details>

>

> Agora vou rodar o lint das regras de novo. Se ele acusar algo, trato como erro real de contrato ou como regra de lint que precisa de escopo explícito.

>

> O lint das regras passou depois da correção do parser. Vou adicionar um script simples para essa validação virar rotina e, em seguida, atacar mais um bloco de botões/integrações que ainda estavam só visuais.

>

> A próxima inconsistência clara é o módulo financeiro/admin usando a coleção `plans` sem contrato nas regras. Vou mapear e fechar isso junto com scripts de validação.

>

> O `plans` já tinha bloco de regra, mas encontrei uma sutileza: `create/update/delete` estavam juntos, e isso torna o delete frágil porque tenta validar `request.resource`. Vou separar as operações e deixar o contrato mais previsível.

>

> <details><summary>Edited 2 files</summary>

>

> - Wrote `./Plataforma-Coroado/firestore.rules` (+8 -1)

> - Wrote `./Plataforma-Coroado/package.json` (+7 -5)

>

> </details>

>

> Vou fechar agora o bloco de “Google Tasks” pastoral: em vez de parecer integração externa incompleta, ele vai usar tarefas internas da plataforma, persistidas no Firestore e vinculadas ao pastoral.

>

> As regras de `tasks` exigem `tag`, `createdBy` igual ao usuário autenticado e `updatedAt`. Vou seguir esse contrato para não criar mais dado que nasce inválido.

>

> Vou trocar rótulos que prometiam sincronização Google por uma fila pastoral real dentro da plataforma. Fica mais honesto e usável agora, e a integração externa pode vir depois sem “botão de fachada”.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/PastorsView.tsx` (+59 -18)

>

> </details>

>

> A lógica de tarefas internas entrou. Vou ajustar agora a interface desse bloco para exibir/criar/concluir tarefas de verdade.

>

> O arquivo tem acentuação real, mas o terminal mostra caracteres distorcidos. Vou usar âncoras sem acento para trocar os blocos da interface com segurança.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/PastorsView.tsx` (+5 -5)

>

> </details>

>

> Os botões principais já apontam para ações reais. Agora vou substituir a mensagem de “precisa conectar Google Tasks” pela lista persistida de tarefas pastorais.

>

> Vou trocar esse estado vazio por uma lista real; isso elimina mais um ponto de “botão que parece funcionar, mas não faz nada”.

>

> Vou ler essas linhas como texto bruto para pegar exatamente os caracteres do arquivo e aplicar o patch sem brigar com encoding.

>

> Agora tenho as linhas reais. Vou aplicar a troca do bloco de tarefas e dos rótulos da agenda.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/PastorsView.tsx` (+37 -9)

>

> </details>

>

> Vou aproveitar para corrigir os botões do financeiro admin: “Exportar Sheets” e “Relatório Docs” abriam páginas genéricas do Google, mas não exportavam nada. Vou transformar em download real de CSV e relatório TXT.

>

> Mesmo padrão de encoding. Vou ler as linhas exatas do financeiro e aplicar em partes.

>

> Agora aplico o export real no financeiro.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/admin/AdminFinance.tsx` (+44 -4)

>

> </details>

>

> Agora vou rodar tipos, regras e build para pegar qualquer regressão dessas mudanças.

>

> Tipos, lint de regras e functions passaram. Vou rodar a build final desta rodada.

>

> Build passou. Vou fazer uma última varredura estática de coleções/regras e registrar o status do repositório.

>

> A varredura final deu `missing: []` para coleções vs regras. Vou atualizar o documento de implementação com o que entrou nesta continuação:
`firebase.json`, lint de regras, tarefas pastorais internas e export financeiro real.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+9 -1)

>

> </details>

>

> Já atualizei o registro técnico. Vou avançar mais um ponto de autonomia: o certificado da Escola IDE ainda estava explicitamente “simulado”; vou transformar em download real de um certificado HTML simples.

>

> O certificado está dentro de uma lista estática de certificados emitidos. Vou manter essa lista por enquanto, mas trocar o botão por um arquivo real que o usuário consegue abrir/imprimir.

>

> Vou adicionar o gerador de certificado como HTML local, com dados escapados e nome de arquivo pelo código do certificado.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+64 -9)

>

> </details>

>

> Agora vou trocar o botão simulado pelo download real.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+3 -3)

>

> </details>

>

> Certificado simulado virou arquivo real baixável. Vou validar tipos de novo.

>

> Tipos e regras passaram; a busca por esses resquícios não retornou mais nada.
Vou rodar a build mais uma vez depois do certificado.

>

> Build passou também. Vou registrar essa última evolução no documento e
coletar o status final.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+1 -0)

>

> </details>

</details>

Continuei e fechei mais uma rodada.

Criei o [firebase.json](<./Plataforma-Coroado/firebase.json>) corretamente para este projeto, com:

- Firestore apontando para `firestore.rules`
- índices em [firestore.indexes.json](<./Plataforma-Coroado/firestore.indexes.json>)
- Functions em `functions/`
- Hosting usando `dist/` com rewrite para SPA
- emuladores de Auth, Firestore, Functions, Hosting e UI

Também validei se ele “existia antes”: não encontrei `firebase.json` no workspace `D:\Projetos\Coroado` nem no histórico Git local desse checkout. Pode ter existido em outro clone/branch remoto, mas neste diretório atual ele não estava presente.

Avancei nos demais pontos:

- Corrigi [eslint.config.js](<./Plataforma-Coroado/eslint.config.js>) para validar `rules` com o parser correto do Firebase.
- Adicionei `lint:rules` e `validate` no [package.json](<./Plataforma-Coroado/package.json>).
- Separei melhor as regras de `plans` no Firestore para `create`, `update` e `delete`.
- Troquei a aba pastoral que prometia “Google Tasks” por tarefas pastorais internas persistidas em `tasks`.
- Troquei os botões falsos de exportação do financeiro admin por download real de CSV e relatório TXT.

- Troquei o download de certificado simulado da Escola IDE por um certificado HTML real, baixável e imprimível.
- Atualizei o registro em [IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md](<./Plataforma-Coroado/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md>).

Validações desta rodada:

- `firebase.json` e `firestore.indexes.json` : JSON válido.
- `npx firebase-tools --version` : Firebase CLI disponível, versão `15.20.0` .
- `npm run lint` : passou.
- `npm run lint:rules` : passou.
- TypeScript das Functions: passou.
- `npm run build` : passou.
- Varredura coleções vs regras: `missing: []` .

A única validação que ainda não deu para concluir foi o emulador oficial do Firestore, porque o Firebase CLI precisa de Java e o ambiente respondeu: Java não está instalado ou não está no PATH. O app continua rodando localmente em `http://localhost:3000/` .

> temos como evoluir nossa plataforma para um stack mais robusta. ajustando codigos pada linguagens como next.js ou react/vue, node.js. tRPC, GraphQL. onde a gente consiga evoluir toda parte de desempenho da plataforma e recursos visuais como transições. motions recursos estéticos e etc

<details><summary>2 previous messages</summary>

> Vou responder com base no estado real da plataforma, não só em preferência de stack. Vou fazer uma checagem rápida do que o projeto já usa hoje para separar “evoluir com segurança” de “reescrever por ansiedade tecnológica”.

```
>  
> <details><summary>Searched the web</summary>  
>  
> <details><summary>Searched the web</summary>  
>  
> - Searched the web for `Next.js official docs App Router Server Components API routes`  
>  
> </details>  
>  
> </details>  
</details>
```

Sim, temos como evoluir bastante. E eu recomendo fazer isso, mas **sem trocar tudo de uma vez**.

Hoje a plataforma está em **React + Vite + Node/Express + Firebase**. Isso já é uma boa base, mas ela cresceu como uma plataforma grande dentro de uma estrutura de app mais simples. O próximo salto saudável seria transformar em uma arquitetura mais modular.

Minha recomendação para a Coroadó:

Stack Alvo

- **Next.js + React + TypeScript** para a aplicação principal.
- **Node.js** como camada backend/BFF.
- **tRPC** para APIs internas da plataforma, porque dá tipagem ponta a ponta sem precisar manter schemas separados.
- **GraphQL apenas depois**, se a plataforma precisar expor API para app mobile, BI, integrações externas ou múltiplos consumidores.

- **Firestore Auth** pode continuar.
- **Firestore + SQL híbrido**: Firestore para tempo real e operação; SQL para financeiro, pedidos, indicadores, auditoria e relatórios.
- **Motion/Tailwind/shadcn/base-ui** para elevar transições, microinterações, layout e experiência visual.

Eu não recomendo migrar para Vue agora. Vue é bom, mas a plataforma já está em React, já usa `motion`, `react-router`, `recharts`, Firebase SDK e componentes React. Migrar para Vue seria uma reescrita mais cara sem ganho proporcional. O caminho natural é **React mais robusto via Next.js**.

Por Que Next.js

Next.js com App Router permite estruturar rotas, layouts, Server Components e Client Components de forma mais madura. Isso ajuda muito numa plataforma como a Coroadó, onde cada página funciona quase como um produto próprio. A documentação oficial do Next destaca justamente esse modelo de App Router e componentes server/client: [Next.js App Router](https://nextjs.org/docs/app) e [Server and Client Components](https://nextjs.org/docs/app/getting-started/server-and-client-components).

tRPC vs GraphQL

Para a Coroadó, eu começaria com **tRPC**. Ele é excelente quando frontend e backend estão em TypeScript, porque cria APIs tipadas sem schema manual nem geração de código pesada. A própria documentação do tRPC resume essa proposta como APIs end-to-end typesafe: [tRPC docs](https://trpc.io/docs/).

GraphQL é poderoso, mas eu deixaria para uma segunda fase. Ele faz mais sentido quando temos vários consumidores da API, integrações externas, app mobile ou necessidade de consultas flexíveis por terceiros. A documentação oficial define GraphQL como linguagem de consulta e runtime com schema fortemente tipado: [GraphQL](https://graphql.org/).

Plano De Evolução

1. **Fase 1: Modularizar sem reescrever**

Separar domínios: membros, células, escola, eventos, financeiro, loja, social, pastoral, admin.

2. **Fase 2: Criar camada API**

Tirar regra de negócio do frontend e levar para Node/tRPC: pagamentos, permissões, relatórios, indicadores, tarefas, inscrições.

3. **Fase 3: Migrar para Next.js**

Começar pelas áreas públicas e dashboards, mantendo Firebase Auth e Firestore no início.

4. **Fase 4: Performance**

Code splitting por módulo, lazy loading, cache, Suspense, imagens otimizadas e carregamento progressivo. React Suspense já dá base para fallback visual enquanto dados/componentes carregam: [React Suspense](https://react.dev/reference/react/Suspense).

5. **Fase 5: Visual Premium**

Transições entre áreas, skeletons, dashboards mais fluidos, animações de estado, timeline de ações, cards com microinterações e navegação mais parecida com produto SaaS moderno.

Minha decisão técnica seria: **Next.js + React + Node + tRPC primeiro; GraphQL depois se houver necessidade real; Vue não agora.** Isso dá robustez, melhora performance e prepara a plataforma para crescer sem virar uma reescrita caótica.

> Faça tbm a implementação do Java para conseguir rodar a validação do firestore

<details><summary>25 previous messages</summary>

> Vou instalar/configurar Java de um jeito que permita rodar o emulador do Firestore sem depender de chute. Primeiro verifico se já existe Java, depois vejo o melhor caminho: instalação pelo gerenciador do Windows ou JDK local no projeto.

>

> Não há Java no PATH. O Windows tem `winget`, então vou tentar instalar um JDK LTS por ele; se a instalação pedir privilégio ou falhar, parto para um JDK portátil local.

>

> O `winget` encontrou o Microsoft OpenJDK 21 e o Temurin 21. Vou instalar o Microsoft OpenJDK 21, que é LTS e suficiente para o emulador do Firestore.

>

> A instalação pelo `winget` demorou demais e estourou o tempo da chamada. Vou verificar se ela concluiu em segundo plano; se não tiver concluído, faço o caminho portátil local para não ficarmos reféns do instalador do Windows.

>

> O instalador ficou pendurado (`winget` / `msiexec`) e ainda não colocou `java` no PATH. Vou encerrar esse instalador travado com cuidado e instalar um JDK portátil dentro do workspace, que resolve a validação sem mexer no sistema inteiro.

>

> Vou usar um JDK portátil em `C:\Users\marco\codex\tools\jdk-21`. Assim não deixamos centenas de MB dentro do repositório, mas a máquina passa a conseguir rodar a validação.

>

> O `winget` foi encerrado, mas um `msiexec` sem acesso continua preso no sistema. Ele não impede o caminho portátil. Vou baixar e extrair o JDK 21 agora.

>

> Download concluído. Vou extrair e apontar `JAVA\_HOME` para a pasta correta do JDK.

>

> A extração funcionou; o comando mostrou OpenJDK 21.0.11, mas o PowerShell marcou erro porque `java -version` escreve em stderr. Vou configurar o ambiente e criar um script de validação para o projeto usar esse JDK automaticamente.

>

> Vou registrar `JAVA\_HOME` e adicionar o `bin` do JDK no PATH do usuário. Também atualizo o PATH da sessão atual para validar imediatamente.

>

> Vou deixar isso reproduzível no repositório: um script de validação do Firestore que usa esse JDK portátil quando necessário e roda o emulador com projeto demo.

>

> <details><summary>Created a file, edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/package.json` (+2 -1)

> - Wrote `./Plataforma-Coroado/scripts/validate-firestore.ps1` (+21 -0)

>

> </details>

>

> Script criado. Agora vou rodar a validação real do Firestore com o emulador, já usando o Java instalado. Pode demorar um pouco porque o Firebase baixa componentes do emulador na primeira execução.

>

> O próprio `java -version` estava sendo tratado como erro pelo PowerShell porque ele escreve a versão em stderr. Vou ajustar o script para imprimir a versão sem falhar e repetir.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/scripts/validate-firestore.ps1` (+1 -1)

>

> </details>

>

> Vou rodar a validação novamente.

>

> Java já está OK. O erro agora é no comando `npx`: ele não resolveu o executável quando chamado pelo script. Vou trocar para a forma que o Firebase CLI reconhece melhor no Windows.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/scripts/validate-firestore.ps1` (+1 -1)

>

> </details>

>

> Comando ajustado. Vou rodar novamente; agora ele deve passar para a etapa do emulador Firestore.

>

> O `npx` está se comportando diferente dentro do script. Vou chamar explicitamente `npx.cmd`, que evita a camada PowerShell ambígua.

>

> Fora do script, o Firebase ainda não via Java porque a sessão não tinha o PATH ajustado. O script ajusta isso, então vou só trocar para `npx.cmd` dentro dele e testar.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/scripts/validate-firestore.ps1` (+1 -1)

>

> </details>

>

> Boa, agora a validação real do Firestore passou com o emulador. Vou rodar o `npm run validate` completo para garantir que o novo fluxo integrado também passa.

>

> A validação completa passou de ponta a ponta. Vou só registrar isso no documento de implementação e verificar se o instalador travado ficou para trás.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+5 -2)

>

> </details>

</details>

Feito. Configurei Java 21 para a validação do Firestore funcionar.

O que ficou implementado:

- Instalei um JDK 21 portátil em `C:\Users\marco\codex\tools\jdk-21`.
- Configurei `JAVA\_HOME` do usuário para esse caminho.
- Criei o script [validate-firestore.ps1](./Plataforma-Coroado/scripts/validate-firestore.ps1) para injetar Java no PATH e rodar o emulador.
- Atualizei [package.json](./Plataforma-Coroado/package.json) com:
 - `validate:firestore`
 - `validate` agora rodando tipos, regras, emulador Firestore e build.
- Atualizei o registro em [IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md).

Validação concluída com sucesso:

- `npm run validate:firestore` : passou.
- `npm run validate` : passou completo.

- Firestore Emulator iniciou corretamente usando OpenJDK `21.0.11`.
- O script retornou `firestore-rules-ok`.

A tentativa inicial via `winget` travou, então usei a solução portátil para não depender do instalador do Windows. Funcionalmente, o bloqueio do Java para validar Firestore está resolvido.

> Sobre o plano de evolução com as 6 fases faça a intersecção dele com o que precisamos seguinha da Auditoria e avance

<details><summary>32 previous messages</summary>

> Vou transformar aquele plano de evolução em uma matriz prática com os pontos da auditoria e, em vez de parar no documento, vou avançar a primeira entrega técnica: criar uma base modular de contratos/serviços que prepare Next/tRPC/SQL sem reescrever tudo agora.

>

> A intersecção mais segura é começar pela Fase 1/2: contratos de domínio e camada de serviço. Isso ataca diretamente a auditoria, prepara tRPC/Next/SQL e evita a armadilha de reescrever a interface antes de estabilizar dados. Vou implementar essa fundação agora.

>

> Vou criar dois pilares pequenos mas importantes: `domain/contracts` para os formatos que futuramente viram tRPC/GraphQL, e `api/http` para tirar fetch solto dos componentes. Em seguida conecto isso nos fluxos de visitante e checkout, que são pontos P0/P1 da auditoria.

>

> Vou editar agora os arquivos centrais: adicionar contratos compartilhados, cliente HTTP tipado e conectar os dois fluxos críticos. Depois valido tudo com `npm run validate`.

>

> <details><summary>Created 3 files</summary>

>

> - Wrote `./Plataforma-Coroado/PLANO\_EVOLUCAO\_STACK\_AUDITORIA\_COROADO\_2026-06-14.md`
(+70 -0)

> - Wrote `./Plataforma-Coroado/src/lib/api/http.ts` (+39 -0)

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+70 -0)

>

> </details>

>

> Agora vou conectar os contratos novos no servidor e nos componentes. Isso já transforma um pedaço da arquitetura em padrão reaproveitável para os próximos módulos.

>

> A primeira edição bateu em diferença de texto no arquivo da Home. Vou aplicar em blocos menores, começando pelo servidor e depois pelos componentes com as linhas exatas.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/server.ts` (+8 -9)

> - Wrote `./Plataforma-Coroado/server.ts` (+9 -9)

>

> </details>

>

> Servidor conectado aos contratos. Vou aplicar agora no frontend usando a leitura exata dos trechos.

>

> Vou trocar os fetches manuais por `postJson` e payload tipado.

>

> <details><summary>Edited 2 files</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/HomeView.tsx` (+14 -21)

> - Wrote `./Plataforma-Coroado/src/components/StoreView.tsx` (+11 -13)

>

> </details>

>

> A primeira fatia da camada de contratos está conectada. Vou rodar tipos agora, porque mudanças compartilhadas costumam revelar pequenos ajustes.

>

> O erro foi só no tipo do helper HTTP: eu modeliei sucesso e falha como interseção sem querer. Vou corrigir a tipagem.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/api/http.ts` (+1 -1)

>

> </details>

>

> Vou rodar a validação de tipos novamente.

>

> Mais um ajuste de TypeScript no retorno genérico do helper. Vou deixar explícita a conversão do payload.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/api/http.ts` (+1 -1)

>

> </details>

>

> Vou rodar `npm run lint` outra vez.

>

> Vou acrescentar também um mapa central de coleções. É pequeno, mas importante: ele vira o “vocabulário oficial” entre Firestore, serviços, futura API tRPC e futura camada SQL.

>

> <details><summary>Created 2 files, edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/server.ts` (+10 -9)

> - Wrote `./Plataforma-Coroado/src/lib/domain/collections.ts` (+40 -0)

> - Wrote `./Plataforma-Coroado/src/lib/domain/index.ts` (+2 -0)

>

> </details>

>

> Mapa de coleções conectado no backend. Vou validar.

>

> Tipos passaram. Vou atualizar o registro da auditoria/evolução e depois rodar a validação completa com regras, emulador e build.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+4 -0)

>

> </details>

>

> Agora vou rodar `npm run validate` completo.

>

> A validação passou, mas a build mostrou um efeito colateral: ao importar o contrato Zod no frontend, o bundle principal cresceu um pouco. Vou separar

“payload leve de cliente” de “validação Zod server-side” para manter a direção de performance.

>

> Vou mover a parte leve para `payloads.ts` e deixar Zod só no servidor/contratos.

>

> <details><summary>Created a file, edited 3 files</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/StoreView.tsx` (+1 -1)

> - Wrote `./Plataforma-Coroado/src/lib/domain/index.ts` (+1 -0)

> - Wrote `./Plataforma-Coroado/src/lib/domain/payloads.ts` (+32 -0)

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+2 -23)

>

> </details>

>

> Agora vou validar e conferir se o bundle volta a ficar mais saudável.

>

> Tipos passaram. Vou rodar a validação completa novamente.

>

> Validação completa passou e o bundle voltou ao patamar anterior, sem carregar Zod no frontend por causa do checkout. Vou recolher o status final e atualizar o plano com essa decisão de performance.

>

> Vou adicionar uma nota no plano: validação pesada fica no servidor, payload leve no cliente. Isso vira regra para as próximas fases.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/PLANO\_EVOLUCAO\_STACK\_AUDITORIA\_COROADO\_2026-06-14.md` (+8 -0)

>
> </details>
</details>

Avancei a interseção do plano de 6 fases com a auditoria e implementei a primeira fatia prática da Fase 1: contratos, payloads e base de API.

Criei o plano em [PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md](<./Plataforma-Coroado/PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md>). Ele cruza cada fase com os problemas da auditoria: contratos, BFF/tRPC, SQL/BI, Next.js gradual, UX/performance e automações.

Também implementei a fundação técnica:

- [collections.ts](<./Plataforma-Coroado/src/lib/domain/collections.ts>): mapa oficial das coleções Firestore.
- [platform-contracts.ts](<./Plataforma-Coroado/src/lib/domain/platform-contracts.ts>): contratos Zod para visitante e checkout.
- [payloads.ts](<./Plataforma-Coroado/src/lib/domain/payloads.ts>): payloads leves para o frontend, sem carregar Zod no bundle.
- [http.ts](<./Plataforma-Coroado/src/lib/api/http.ts>): cliente HTTP tipado para preparar a futura camada BFF/tRPC.
- [server.ts](<./Plataforma-Coroado/server.ts>): passou a validar `visitor-leads` e `checkout` pelos contratos.
- [HomeView.tsx](<./Plataforma-Coroado/src/components/HomeView.tsx>): lead público agora usa `postJson`.
- [StoreView.tsx](<./Plataforma-Coroado/src/components/StoreView.tsx>): checkout agora envia payload mínimo (`productId`, quantidade, tamanho/cor), preparando o backend como fonte real de preço.

Ponto importante: eu separei validação pesada de payload leve. Zod fica no servidor/contratos; o frontend usa apenas helpers pequenos. Isso mantém a direção de performance antes de migrarmos para Next/tRPC.

Validação final:

- `npm run validate` : passou completo.
- Inclui TypeScript, lint das regras, Firestore Emulator com Java 21 e build Vite.

Próximo bloco natural: extrair contratos para `members`, `cell\_reports`, `events/event\_enrollments` e `transactions/orders/payments`. Esse é o caminho direto para depois plugar tRPC sem reescrever a plataforma inteira no escuro.

> Perfeito. Pode avançar

<details><summary>50 previous messages</summary>

> Vou avançar pelo próximo bloco mais valioso: contratos dos domínios que viram indicadores e fluxo financeiro/eventos. Vou manter a mesma lógica: contratos fortes no servidor/shared, payload leve no cliente e validação completa no fim.

>

> Context compaction

>

> Source: automatic

> Status: completed

>

> Vou avançar no próximo bloco: transformar os dados recorrentes mais críticos em contratos reutilizáveis e ligar isso a fluxos reais, começando por células e finanças. Esse é um bom passo porque já prepara os futuros indicadores sem exigir uma migração grande agora.

>

> Já encontrei os pontos de escrita mais urgentes: relatórios de células e transações financeiras ainda montam os dados diretamente na tela. Vou centralizar esses formatos agora para reduzir inconsistência e deixar pronto para indicadores.

>

> Vou fazer uma alteração pequena mas estrutural: em vez de cada tela “inventar” o formato do dado, ela passa a usar uma fábrica de payload com os campos obrigatórios, nomes consistentes e limites básicos. Isso mantém a experiência simples para quem usa, mas melhora bastante a sustentação técnica por baixo.

>

> Também vou revisar um detalhe de regras: alguns documentos têm campos extras úteis, mas a regra só exige o núcleo mínimo. Os contratos vão preservar esse núcleo obrigatório e aceitar os anexos operacionais, como filhos no evento ou dados do visitante na célula.

>

> A base já tem um catálogo central de coleções, então vou reaproveitá-lo em vez de criar outro caminho. Esse tipo de escolha ajuda a manter a evolução do stack mais gradual e menos traumática.

>

> Antes de editar, confirmei que o frontend já importa apenas os helpers leves quando precisa enviar dados. Vou manter assim para não empurrar validação pesada para o pacote do navegador.

>

> Agora vou editar quatro pontos: contratos centrais, helpers leves, células e finanças. Também vou incluir inscrições de eventos no mesmo padrão, porque esse fluxo cruza presença, pagamento, check-in e capacidade.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/payloads.ts` (+102 -0)

>

> </details>

>

> Os helpers de payload já foram criados. Eles limpam textos, normalizam IDs, evitam duplicidade de presença e garantem que contribuição financeira sempre tenha status, data, método e tenant.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+97 -0)

>

> </details>

>

> Contratos centrais adicionados. Vou ajustar um detalhe de compatibilidade da validação numérica antes de ligar as telas, para o build não depender de um comportamento específico da versão do validador.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+2 -2)

>

> </details>

>

> Vou conectar primeiro o relatório de células, que era um dos pontos citados na auditoria e tem alto valor como indicador futuro: frequência, visitantes, consistência de encontros e acompanhamento por célula.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+10 -8)

>

> </details>

>

> Agora vou aplicar o mesmo padrão no financeiro, principalmente para contribuições via Pix manual. A tela continua simples, mas o registro já fica padronizado para conciliação e indicadores.

>

> A primeira tentativa de patch esbarrou na codificação de acentos do arquivo, então vou fazer a alteração por trechos menores, sem mexer em textos existentes além do necessário.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/FinanceView.tsx` (+5 -3)

>

> </details>

>

> Importações de suporte conectadas no financeiro. Vou substituir só a montagem do documento de transação agora.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/FinanceView.tsx` (+24 -16)

>

> </details>

>

> Transações Pix agora passam por um payload único e validam valor antes de registrar. Vou fazer o mesmo na inscrição de eventos, porque ela precisa conversar com pagamento e check-in com menos risco de dados quebrados.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+5 -3)

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+21 -21)

>

> </details>

>

> Encontrei uma sutileza técnica nos contratos numéricos: alguns campos precisam ser inteiros depois de aplicar limites. Vou ajustar isso agora para manter a tipagem limpa.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+2 -3)

>

> </details>

>

> Com os fluxos conectados, vou rodar a validação completa agora. Isso deve pegar rapidamente problemas de tipo, build e regras do Firestore.

>

> Validação completa passou: tipos, regras, emulador do Firestore com Java 21 e build de produção. Vou registrar agora no material da auditoria/plano para manter a rastreabilidade do que já saiu do diagnóstico e virou implementação.

>

> Vou atualizar os documentos de execução sem reescrever a auditoria inteira: só acrescentar o bloco novo e os artefatos alterados. Assim ela continua servindo como trilha de decisões.

>

> Vou marcar no plano que a Fase 1 avançou além de checkout/lead: agora inclui os domínios recorrentes que viram indicador. Isso deixa claro o que já está pronto para a próxima etapa de backend/BFF.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+5 -0)

>

> </details>

>

> Documento de implementação atualizado. Agora vou ajustar o plano de evolução para refletir que os contratos de células, finanças e eventos deixaram de ser “próximos” e já entraram na base.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/PLANO\_EVOLUCAO\_STACK\_AUDITORIA\_COROADO\_2026-06-14.md` (+12 -3)

>

> </details>

>

> A trilha documental está alinhada. Vou fazer uma leitura final das alterações para checar se o escopo ficou limpo e se não surgiu nada fora do combinado.

>

> Como a validação passou, vou avançar mais um degrau da Fase 1 para a Fase 2: criar serviços de domínio para que as telas parem de conhecer tantos detalhes de gravação no Firestore. É o mesmo caminho que depois vira BFF/tRPC com menos retrabalho.

>

> Vou adicionar três serviços pequenos e objetivos: relatórios de célula, contribuições e inscrições de evento. Eles ainda usam Firestore por baixo, mas isolam a regra de escrita.

>

> <details><summary>Created 4 files</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/services/cellReportsService.ts` (+14 -0)

> - Wrote `./Plataforma-Coroado/src/lib/services/eventEnrollmentsService.ts`
(+31 -0)

> - Wrote `./Plataforma-Coroado/src/lib/services/financeService.ts` (+14 -0)

> - Wrote `./Plataforma-Coroado/src/lib/services/index.ts` (+3 -0)

>

> </details>

>

> Serviços criados. Agora vou limpar as telas para chamarem esses serviços,
reduzindo acoplamento sem alterar a experiência do usuário.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+3 -9)

>

> </details>

>

> Células agora usam serviço de domínio para gravar relatórios. Vou ajustar
finanças do mesmo jeito.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/FinanceView.tsx` (+6 -12)

>

> </details>

>

> Finanças agora também saiu da montagem manual de transações. Faltam eventos, onde o ganho maior é deixar a inscrição e o vínculo com pagamento menos espalhados.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+17 -24)

>

> </details>

>

> Eventos agora usam serviço de inscrição. Vou fazer uma checagem rápida de imports sobrando e depois rodar a validação novamente.

>

> A checagem mostrou só um comentário que ficou antigo no fluxo de evento. Vou limpar esse detalhe para o código contar a verdade.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+6 -6)

>

> </details>

>

> Agora vou rodar a validação completa de novo com a camada de serviços incluída.

>

> A validação completa passou novamente. Vou registrar a camada de serviços no plano, junto com o resultado da validação.

>

> Vou fazer um último patch documental curto e depois fecho este ciclo com o resumo objetivo do que foi implementado e validado.

```
>
> <details><summary>Edited 2 files</summary>
>
> - Wrote `./Plataforma-
Coroado/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md` (+2 -0)
> - Wrote `./Plataforma-
Coroado/PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md`
(+3 -2)
>
> </details>
>
> Documentação atualizada. Vou só conferir o estado final dos arquivos tocados
neste ciclo para garantir que o pacote da entrega está claro.
</details>
```

Avancei mais um bloco da evolução stack + auditoria.

Implementei contratos e serviços de domínio para dados recorrentes que viram indicadores: células, finanças, eventos, pedidos e membros. As telas agora deixam de montar parte desses dados “na mão” e passam por payloads/serviços reutilizáveis.

Principais arquivos:

- [platform-contracts.ts](./Plataforma-Coroado/src/lib/domain/platform-contracts.ts:60)
- [payloads.ts](./Plataforma-Coroado/src/lib/domain/payloads.ts:31)
- [cellReportsService.ts](./Plataforma-Coroado/src/lib/services/cellReportsService.ts:1)
- [financeService.ts](./Plataforma-Coroado/src/lib/services/financeService.ts:1)
- [eventEnrollmentsService.ts](./Plataforma-Coroado/src/lib/services/eventEnrollmentsService.ts:1)

Conectei isso em:

- Relatórios de células: presença, visitantes e resumo padronizados.
- Financeiro: contribuição Pix manual registrada como pendente, com valor validado.
- Eventos: inscrição padronizada, incluindo filhos, pagamento e preferência Mercado Pago.

Também atualizei:

- [IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md:1)
- [PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md:1)

Validação completa passou novamente com `npm run validate`, incluindo TypeScript, lint das regras, emulador Firestore com Java 21 e build de produção.

Próximo bloco natural: expandir `src/lib/services/` para membros, tarefas, pastoral e social, e então começar a primeira rota BFF/tRPC nos fluxos que precisam de regra server-side.

> Maravilha, avance

<details><summary>3 previous messages</summary>

> Perfeito, vou avançar no próximo bloco da trilha: levar a mesma organização para membros, tarefas, pastoral e social. A ideia é reduzir mais regras espalhadas nas telas antes de começar a camada BFF/tRPC de verdade.

>

> Vou começar lendo os pontos em que essas telas gravam dados hoje. Assim a mudança entra onde realmente reduz risco, sem uma refatoração decorativa.

>

> Já apareceram quatro blocos claros: edição de membro, agendamento pastoral, tarefas pastorais e agendamento social. Vou aprofundar nesses trechos para criar serviços que respeitem os campos que as regras do Firestore esperam.

</details>

A leitura confirmou um ganho imediato extra: algumas atualizações importantes ainda não enviavam `updatedAt`, e isso é o tipo de detalhe que quebra regra do Firestore ou reduz rastreabilidade. Vou corrigir isso dentro dos serviços, não espalhado em cada tela.

> continue

<details><summary>29 previous messages</summary>

> Continuo sim. Vou fechar esse bloco criando os serviços restantes e ligando nas telas, depois rodo a validação completa de novo.

>

> Também vou corrigir um ponto operacional no Social: a lista de atendimentos estava lendo todos os agendamentos sem filtro de unidade/tenant. Vou amarrar isso ao tenant para evitar mistura de dados entre áreas.

>

> Vou criar os novos builders de payload primeiro. Eles são leves e continuam próprios para frontend, enquanto os contratos Zod ficam disponíveis para a próxima etapa server-side.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/payloads.ts` (+139 -0)

>

> </details>

>

> Builders criados. Vou ampliar os contratos centrais para cobrir esses novos documentos e status recorrentes.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+74 -0)

>

> </details>

>

> Agora entram os serviços de domínio: membros, pastoral e social. Eles concentram criação/atualização, timestamps e nomes de coleção.

>

> <details><summary>Created 3 files, edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/services/index.ts` (+3 -0)

> - Wrote `./Plataforma-Coroado/src/lib/services/membersService.ts` (+22 -0)

> - Wrote `./Plataforma-Coroado/src/lib/services/pastoralService.ts` (+60 -0)

> - Wrote `./Plataforma-Coroado/src/lib/services/socialService.ts` (+52 -0)

>

> </details>

>

> Serviços criados. Vou conectar primeiro Membros, que é sensível porque mistura perfil, célula, ministério e permissões.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/MembersView.tsx` (+21 -23)

>

> </details>

>

> Membros já usa serviço. Agora vou tirar do Pastores a criação direta de agendamento e tarefas pastorais, mantendo a gestão de cadastro de pastores como está por enquanto.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/PastorsView.tsx` (+25 -34)

>

> </details>

>

> Vou conectar também a área de cuidado pastoral: visitantes e pedidos de oração deixam de atualizar status diretamente pela tela.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/PastoralCareView.tsx` (+28 -29)

>

> </details>

>

> Agora vou para o Social. Além do serviço, vou amarrar consultas administrativas ao tenant para evitar leitura ampla demais.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SocialView.tsx` (+21 -27)

>

> </details>

>

> Vou trocar os nomes literais de coleção restantes nesse trecho por constantes centrais quando já estamos mexendo nele. Isso ajuda a evitar divergência silenciosa de nomes no futuro.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SocialView.tsx` (+2 -2)

>

> </details>

>

> Vou fazer uma varredura rápida por imports sobrando e escritas diretas que deveriam ter saído desse bloco.

>

> Encontrei um detalhe no vocabulário de pagamento: o Social usa `paid`, então vou incluir esse status no contrato compartilhado antes de validar.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+1 -1)

>

> </details>

>

> Vou rodar a validação agora; se aparecer algo de tipo ou regra, eu corrijo na sequência.

>

> Validação passou novamente. Vou atualizar os documentos de auditoria/plano com esse segundo bloco de serviços.

>

> Vou registrar exatamente o que entrou: serviços de membros, pastoral e social; contratos novos; e o filtro por tenant nos atendimentos sociais.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+7 -0)

>

> </details>

>

> Documento de implementações atualizado. Vou ajustar também o plano de evolução para mostrar que a Fase 2 já começou de forma incremental.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/PLANO\_EVOLUCAO\_STACK\_AUDITORIA\_COROADO\_2026-06-14.md` (+4 -1)

>

> </details>

>

> Vou fazer uma checagem final de status e dos arquivos tocados neste ciclo antes de fechar o resumo.

>

> Como este trabalho faz parte da sequência longa da Coroadó, vou só consultar a memória do projeto para manter a referência correta no fechamento.

</details>

Avancei mais um bloco da evolução.

Agora a plataforma tem serviços de domínio também para membros, pastoral e social:

- [membersService.ts](./Plataforma-Coroado/src/lib/services/membersService.ts:1)
- [pastoralService.ts](./Plataforma-Coroado/src/lib/services/pastoralService.ts:1)
- [socialService.ts](./Plataforma-Coroado/src/lib/services/socialService.ts:1)

Conectei esses serviços em:

- [MembersView.tsx](./Plataforma-Coroado/src/components/MembersView.tsx:17): edição de dados do membro e sincronização de líder de ministério.
- [PastorsView.tsx](./Plataforma-Coroado/src/components/PastorsView.tsx:12): agendamento pastoral e tarefas pastorais.
- [PastoralCareView.tsx](./Plataforma-Coroado/src/components/PastoralCareView.tsx:9): status de visitantes e pedidos de oração.
- [SocialView.tsx](./Plataforma-Coroado/src/components/SocialView.tsx:13): agendamentos sociais, profissionais sociais e filtro por `tenantId`.

Também ampliei os contratos em [platform-contracts.ts](./Plataforma-Coroado/src/lib/domain/platform-contracts.ts:138) e os builders em [payloads.ts](./Plataforma-Coroado/src/lib/domain/payloads.ts:126).

Validação completa passou novamente com `npm run validate`: TypeScript, regras Firestore, emulador com Java 21 e build de produção.

Atualizei a trilha em:

- [IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md:1)

- [PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md:1)

Próximo avanço natural: escola, ministérios, escalas e planejamento, depois começar as primeiras rotas BFF/tRPC onde houver regra sensível ou integração externa.

> Perfeito. Avance

<details><summary>38 previous messages</summary>

> Vou avançar no bloco seguinte que ficou natural: escola, ministérios, escalas e planejamento. Antes de mexer, vou mapear onde essas áreas ainda escrevem direto no Firestore para atacar os pontos de maior impacto sem abrir uma refatoração gigante demais de uma vez.

>

> Vou fazer também uma consulta curta na memória do projeto só para manter o alvo correto: este trabalho segue no Firebase/Al Studio da `Plataforma-Coroado`, não na variante Supabase.

>

> O mapa mostrou muitos pontos, então vou dividir a entrega para não ficar arriscado: primeiro escola e planejamento/tarefas, depois ministérios/escalas. Começo pelos contratos que são mais fáceis de reutilizar em BFF/tRPC depois.

>

> Vou ler agora os trechos de planejamento e ministérios. Essa parte tem vários fluxos parecidos com tarefas e briefings, então dá para ganhar bastante consistência com poucos serviços bons.

>

> Agora vou adicionar os builders para escola, planejamento e ministérios. Eles vão padronizar campos mínimos exigidos pelas regras, mas ainda preservam campos extras que as telas já usam.

>

> Vou fazer essa etapa em duas camadas, como nas anteriores: primeiro os payloads/contratos; depois os serviços que gravam. Assim o frontend continua leve e o futuro backend já encontra os formatos prontos.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/payloads.ts` (+257 -0)

>

> </details>

>

> Payloads prontos. Vou acrescentar os contratos Zod equivalentes, para deixar a futura camada server-side com as mesmas fronteiras de dados.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+100 -0)

>

> </details>

>

> Agora vou criar os serviços: `schoolService`, `planningService` e `ministriesService`. Eles vão centralizar as gravações e reaproveitar os builders recém-criados.

>

> <details><summary>Created 3 files, edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/services/index.ts` (+3 -0)

> - Wrote `./Plataforma-Coroado/src/lib/services/ministriesService.ts` (+65 -0)

> - Wrote `./Plataforma-Coroado/src/lib/services/planningService.ts` (+48 -0)

> - Wrote `./Plataforma-Coroado/src/lib/services/schoolService.ts` (+70 -0)

>

> </details>

>

> Percebi uma colisão de nome antes de conectar: já havia `updateTaskStatus` no serviço pastoral. Vou nomear a função do planejamento explicitamente como `updatePlanningTaskStatus` para evitar ambiguidade.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/services/planningService.ts` (+1 -1)

>

> </details>

>

> Vou conectar o planejamento Kanban agora: criação, movimentação, edição e comentários de tarefas passam pelo serviço.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/AdminPlanningKanban.tsx` (+53 -58)

>

> </details>

>

> Planejamento conectado. Vou para Escola: módulos, aulas, cursos, trilhas, matrículas e progresso.

>

> Vou aplicar a conexão em pontos bem específicos da Escola para preservar a tela atual: os formulários continuam iguais, mas quem grava é o serviço.

>

> O patch não encontrou uma linha de import porque esse arquivo tem uma organização diferente no topo. Vou ajustar olhando exatamente os imports atuais.

>

> Vou corrigir os imports da Escola e depois trocar as chamadas.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+17 -8)

>

> </details>

>

> Imports ajustados. Agora substituo os blocos de criação/atualização por serviços.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+43 -51)

>

> </details>

>

> Vou trocar também o vínculo curso-trilha e matrícula/progresso por serviços.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+1 -4)

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+10 -19)

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+0 -4)

>

> </details>

>

> Escola já saiu das gravações diretas. Agora vou conectar Ministérios, escalas e briefings aos serviços.

>

> Vou limpar os imports e ligar as funções novas no arquivo de Ministérios.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/MinistriesView.tsx` (+12 -5)

> - Wrote `./Plataforma-Coroado/src/components/MinistriesView.tsx` (+8 -10)

> - Wrote `./Plataforma-Coroado/src/components/MinistriesView.tsx` (+11 -15)

>

> </details>

>

> Vou substituir os botões de status de briefing e escala por chamadas ao serviço.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/MinistriesView.tsx` (+15 -15)

> - Wrote `./Plataforma-Coroado/src/components/MinistriesView.tsx` (+5 -5)

>

> </details>

>

> Agora vou trocar a criação de briefing e de escala.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/MinistriesView.tsx` (+16 -20)

>

> </details>

>

> As três áreas já estão conectadas aos serviços. Vou rodar a validação completa agora para capturar qualquer desalinhamento de tipo, build ou regras.

>

> Validação passou. Vou registrar essa entrega nos documentos da auditoria e plano, para manter a trilha de avanço das fases.

>

> Vou atualizar os documentos com o que foi entregue neste bloco: serviços de escola, planejamento e ministérios/escalas, além da validação completa.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+7 -0)

>

> </details>

>

> Documento de implementações atualizado. Vou ajustar o plano para marcar essa fase como avançada e apontar o próximo corte para BFF/tRPC.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/PLANO\_EVOLUCAO\_STACK\_AUDITORIA\_COROADO\_2026-06-14.md` (+5 -2)

>

> </details>

>

> Vou fazer uma checagem final do estado dos arquivos deste bloco e fechar com um resumo enxuto.

</details>

Avancei mais um bloco da Fase 1/Fase 2.

Agora também existem serviços de domínio para:

- [schoolService.ts](./Plataforma-Coroado/src/lib/services/schoolService.ts:1): cursos, módulos, aulas, trilhas, matrículas e progresso.
- [planningService.ts](./Plataforma-Coroado/src/lib/services/planningService.ts:1): tarefas do Kanban e comentários.
- [ministriesService.ts](./Plataforma-Coroado/src/lib/services/ministriesService.ts:1): ministérios, escalas e briefings.

Conectei esses serviços em:

- [SchoolView.tsx](./Plataforma-Coroado/src/components/SchoolView.tsx:29)
- [AdminPlanningKanban.tsx](./Plataforma-Coroado/src/components/AdminPlanningKanban.tsx:10)
- [MinistriesView.tsx](./Plataforma-Coroado/src/components/MinistriesView.tsx:10)

Também ampliei os contratos/builders em:

- [payloads.ts](./Plataforma-Coroado/src/lib/domain/payloads.ts:274)
- [platform-contracts.ts](./Plataforma-Coroado/src/lib/domain/platform-contracts.ts:226)

Validação completa passou de novo com `npm run validate`: TypeScript, lint das regras, Firestore Emulator com Java 21 e build de produção.

Atualizei a trilha em:

- [IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md:1)

- [PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md:1)

Próximo avanço mais valioso: começar as primeiras rotas BFF/tRPC para fluxos sensíveis, especialmente checkout, contribuições, matrículas, eventos e conciliação financeira.

> Siga ao próximo passo natural

<details><summary>15 previous messages</summary>

> Vou seguir para o próximo passo natural: começar a tirar fluxos sensíveis do cliente e levar para o BFF/servidor. Vou começar por contribuições financeiras, porque é um fluxo crítico, vira indicador e precisa de autenticação/conciliação mais forte.

>

> Vou ler o servidor e o cliente HTTP atual para reaproveitar o padrão já criado, em vez de inventar uma segunda forma de chamar API.

>

> O BFF já existe em Express e já atende checkout com Firebase Auth. Vou seguir esse mesmo caminho agora para contribuições: rota autenticada no servidor, validação central e criação da transação com Admin SDK.

>

> Vou adicionar primeiro o contrato de entrada dessa rota. Ele é menor que o documento Firestore final: a tela informa valor/tipo, e o servidor completa usuário, tenant, status e datas.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+13 -0)

>

> </details>

>

> Contrato criado. Agora vou expor a rota `/api/contributions` no servidor, autenticada pelo Firebase e gravando a transação com tenant do perfil do usuário.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/server.ts` (+35 -0)

>

> </details>

>

> Rota criada no BFF. Vou conectar o Financeiro para usar essa rota com token da sessão, mantendo o mesmo botão e a mesma mensagem para o usuário.

>

> Vou substituir a gravação direta por chamada autenticada ao BFF. Isso deixa o Firestore client só como leitura/listagem desse histórico, que é menos sensível.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/FinanceView.tsx` (+12 -8)

>

> </details>

>

> A conexão do Financeiro foi feita. Vou rodar uma checagem rápida para garantir que não ficou import antigo nem rota sem contrato, depois valido tudo.

>

> A checagem está limpa. Vou rodar a validação completa agora; se passar, atualizo os documentos da auditoria/plano com essa primeira rota BFF financeira.

>

> Validação passou. Vou registrar essa entrega como o início prático da camada BFF: contribuição financeira já não depende mais de escrita direta do cliente.

>

> <details><summary>Edited 2 files</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+3 -0)

> - Wrote `./Plataforma-Coroado/PLANO\_EVOLUCAO\_STACK\_AUDITORIA\_COROADO\_2026-06-14.md` (+4 -1)

>

> </details>

>

> Vou fazer uma última checagem de arquivos tocados e encerrar esse passo com o que ficou pronto.

</details>

Avancei para o próximo passo natural: primeira rota BFF financeira.

O que entrou:

- Criei o contrato `contributionRequestSchema` em [platform-contracts.ts](./Plataforma-Coroado/src/lib/domain/platform-contracts.ts:89).

- Criei a rota autenticada `POST /api/contributions` em [server.ts](./Plataforma-Coroado/server.ts:327).

- Atualizei o Financeiro para registrar contribuição pelo BFF com token Firebase em [FinanceView.tsx](./Plataforma-Coroado/src/components/FinanceView.tsx:88).

Agora a contribuição Pix pendente não depende mais de escrita sensível direto do cliente. O servidor valida valor/tipo, usa o usuário autenticado, pega `tenantId` do perfil e grava a transação via Admin SDK.

Também atualizei:

- [IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md:17)
- [PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md:64)

Validação completa passou com `npm run validate`: TypeScript, regras Firestore, emulador com Java 21 e build de produção.

Próximo passo natural agora: separar `server.ts` em módulos de API por domínio e expandir BFF para matrículas/eventos/conciliação.

> Perfeito. Avance ao proximo passo natural. Va dando clareza do que falta tbm ao final da entrega

<details><summary>15 previous messages</summary>

> Perfeito. Vou avançar em duas frentes conectadas: organizar o BFF em módulos por domínio e mover matrícula da Escola IDE para o backend. No fim eu deixo bem claro o que foi entregue e o que ainda falta para fechar essa evolução com maturidade.

>

> Vou checar primeiro o formato atual do servidor e da matrícula para fazer essa separação sem mudar comportamento que já está validado.

>

> Vou separar o servidor agora em módulos pequenos: contexto/autenticação, rotas públicas, admin, checkout, financeiro, escola, webhook e notificações. Isso prepara tRPC sem obrigar a migração completa neste momento.

>

> <details><summary>Created 8 files</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/context.ts` (+99 -0)

> - Wrote `./Plataforma-Coroado/src/server/routes/adminRoutes.ts` (+62 -0)

> - Wrote `./Plataforma-Coroado/src/server/routes/checkoutRoutes.ts` (+139 -0)

> - Wrote `./Plataforma-Coroado/src/server/routes/financeRoutes.ts` (+46 -0)

> - Wrote `./Plataforma-Coroado/src/server/routes/mercadoPagoRoutes.ts` (+57 -0)

> - Wrote `./Plataforma-Coroado/src/server/routes/notificationRoutes.ts` (+18 -0)

> - Wrote `./Plataforma-Coroado/src/server/routes/publicRoutes.ts` (+36 -0)

> - Wrote `./Plataforma-Coroado/src/server/routes/schoolRoutes.ts` (+67 -0)

>

> </details>

>

> Os módulos do BFF foram criados. Agora vou trocar o `server.ts` grande por um bootstrap mais limpo que apenas registra as rotas por domínio e serve o frontend.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/server.ts` (+51 -0)

>

> </details>

>

> Agora vou adicionar o contrato específico de matrícula e ligar a Escola IDE nessa nova rota.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+5 -0)

>

> </details>

>

> Vou ajustar a Escola para chamar `/api/school/enrollments` com o token Firebase, removendo a criação direta da matrícula pelo cliente.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+11 -15)

>

> </details>

>

> Vou rodar uma checagem estática rápida antes da validação completa, porque mover o servidor para módulos costuma revelar imports ou tipos que precisam de ajuste fino.

>

> A estrutura está no lugar. Vou validar agora; se o TypeScript reclamar de algum detalhe de módulo, resolvo antes de seguir para documentação.

>

> Validação passou. Vou registrar a entrega nos documentos e deixar uma lista clara do que ainda falta depois deste bloco.

>

> <details><summary>Edited 2 files</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+4 -0)

> - Wrote `./Plataforma-Coroado/PLANO\_EVOLUCAO\_STACK\_AUDITORIA\_COROADO\_2026-06-14.md` (+5 -2)

>

> </details>

>

> Vou fazer uma checagem final dos arquivos tocados e já fecho com o resumo da entrega e o mapa do que falta.

</details>

Avancei o próximo passo natural: o BFF agora começou a ficar organizado por domínio, e matrícula da Escola IDE saiu da escrita direta do cliente.

Entregue:

- `server.ts` virou um bootstrap limpo que só registra rotas e serve o app.
- Criei a estrutura [src/server](./Plataforma-Coroado/src/server/context.ts:1) com autenticação/contexto e rotas por domínio.
- Criei a rota BFF [POST /api/school/enrollments](./Plataforma-Coroado/src/server/routes/schoolRoutes.ts:12).
- A Escola IDE agora chama o BFF em [SchoolView.tsx](./Plataforma-Coroado/src/components/SchoolView.tsx:1896).
- Adicionei o contrato `schoolEnrollmentRequestSchema` em [platform-contracts.ts](./Plataforma-Coroado/src/lib/domain/platform-contracts.ts:263).

A matrícula agora valida curso, usa o usuário autenticado, respeita `tenantId`, evita matrícula duplicada e grava via Admin SDK no backend.

Validação completa passou com `npm run validate`: TypeScript, regras Firestore, emulador Firestore com Java 21 e build de produção.

Também atualizei:

- [IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md:18)
- [PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md:66)

****O Que Falta****

- Mover inscrições de eventos para BFF, principalmente eventos pagos.
- Criar conciliação financeira mais forte para `transactions`, `orders` e pagamentos.
- Adicionar camada formal de autorização server-side por domínio.
- Começar tRPC em cima dessas rotas já modularizadas.
- Definir modelo SQL/BI para indicadores financeiros, células, eventos e escola.
- Depois, migrar progressos/ações administrativas da Escola para servidor quando envolver permissão sensível.

> Avance para finalizar

<details><summary>60 previous messages</summary>

> Vou avançar para fechar os pontos que ficaram como “falta”: inscrições de eventos pelo BFF e uma base de conciliação financeira server-side. Vou manter o foco em terminar com segurança: mover fluxos sensíveis, validar tudo e deixar claro o que ainda fica como evolução estrutural maior.

>

> Vou fazer uma leitura rápida dos trechos exatos antes de editar, para não quebrar o que já passou nas validações anteriores.

>

> Vou mover agora a inscrição de eventos para o BFF. Para evento pago, o backend vai criar a inscrição pendente, abrir a preferência do Mercado Pago e devolver o link de pagamento; para evento gratuito, confirma direto.

>

> Como a tela ainda mostra o componente de pagamento para ingressos pendentes, vou manter essa parte visual. O que muda é a criação da inscrição e da preferência: sai do cliente e vai para o backend.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+15 -0)

>

> </details>

>

> Vou criar a rota de eventos no BFF e registrá-la no servidor.

>

> <details><summary>Created a file, edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/server.ts` (+2 -0)

> - Wrote `./Plataforma-Coroado/src/server/routes/eventRoutes.ts` (+156 -0)

>

> </details>

>

> Agora vou conectar a tela de Eventos a essa rota, removendo a criação direta da inscrição e a chamada antiga da função de preferência.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+30 -43)

>

> </details>

>

> Eventos agora usa BFF para inscrição. Vou adicionar a conciliação financeira server-side para o admin confirmar ou rejeitar transações pendentes sem editar direto no cliente.

>

> Vou manter essa conciliação simples e auditável: o admin muda o status via endpoint autenticado, e o backend grava quem conciliou, quando e uma observação curta.

>

> <details><summary>Edited 2 files</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+6 -0)

> - Wrote `./Plataforma-Coroado/src/server/routes/financeRoutes.ts` (+47 -1)

>

> </details>

>

> Context compaction

>

> Source: automatic

> Status: completed

>

> Vou fechar essa rodada como uma entrega de finalização: amarrar os fluxos que ainda ficaram na borda entre tela e backend, rodar a validação completa e deixar claro o que fica como próxima etapa estrutural, não como pendência escondida.

>

> Já localizei a base ativa e os pontos finais desta passada. O principal ajuste agora é garantir que a conciliação financeira use o mesmo caminho simples que o frontend já usa para chamadas autenticadas, sem deixar a tela administrativa só com botão decorativo.

>

> Encontrei uma pequena incompatibilidade antes de fechar: o utilitário de chamadas da plataforma trabalha hoje em `POST`, enquanto a rota de conciliação tinha sido criada como atualização direta. Vou alinhar isso e conectar a tela financeira com registro de auditoria no backend.

>

> Também notei um detalhe do projeto: alguns aliases apontam para uma pasta `src` interna e outros para componentes na raiz. Vou respeitar essa organização atual para não mexer em estrutura além do necessário.

>

> Vou fazer a edição em três lugares: rota financeira, tela de administração financeira e documentação de entrega. Depois disso rodo a validação completa para saber se fechamos com segurança.

>

> Vou preservar o desenho incremental: em vez de iniciar uma migração grande agora, fecho os últimos fluxos críticos dentro do BFF modular que já foi montado. Isso deixa a plataforma mais segura hoje e pronta para Next/tRPC depois.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/financeRoutes.ts` (+3 -2)

>

> </details>

>

> A rota de conciliação agora ficou alinhada ao cliente HTTP existente e ao dono global configurado no backend. Vou ligar isso na tela do financeiro administrativo com ações explícitas para confirmar ou rejeitar transações pendentes.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/admin/AdminFinance.tsx` (+44 -3)

>

> </details>

>

> Como essa conciliação passa a registrar estados além de “pendente” e “concluído”, vou atualizar o tipo compartilhado da transação para a própria interface aceitar esses resultados sem forçar a tela a fingir que só existem dois estados.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/types/index.ts` (+1 -1)

>

> </details>

>

> Agora vou registrar essa finalização nos documentos de auditoria/plano, para que a entrega fique rastreável e para separar o que foi resolvido do que ainda pertence às próximas fases estruturais.

>

> A documentação está com alguns caracteres herdados em codificação antiga, então vou fazer a atualização usando pontos de ancoragem mais estáveis para evitar mexer no restante do texto.

>

> A atualização do registro de entrega vai ficar em linguagem simples, sem reformatar o documento inteiro. Assim mantemos o histórico e adicionamos só o que mudou nesta última rodada.

>

> Também vou ajustar o plano: a próxima entrega recomendada não pode mais dizer que “inscrição de eventos e conciliação financeira” são o próximo passo, porque acabamos de mover esses dois para implementação.

>

> Para evitar brigar com a codificação antiga do arquivo, vou acrescentar um bloco de “finalização” no próprio documento. Ele passa a ser a leitura mais recente da entrega, sem depender de trocar linhas antigas.

>

> Vou conferir o começo do arquivo em leitura binária/textual curta para encontrar um ponto de inserção estável. É só para ancorar a alteração sem regravar o documento inteiro.

>

> O arquivo em si está correto; quem mostrou caracteres estranhos foi só a saída do terminal. Vou aplicar a alteração usando o texto real em português.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-

Coroado/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md` (+8 -0)

>

> </details>

>

> Vou fazer o mesmo no plano de evolução: deixar um bloco de estado final no topo, mostrando que evento e conciliação saíram da lista de “próximos” e viraram entrega feita.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-

Coroado/PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md`
(+8 -0)

>

> </details>

>

> Ainda há uma frase antiga no plano dizendo que evento/conciliação seriam a próxima entrega. Vou trocar essa linha para não deixar uma contradição no documento.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-

Coroado/PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md`
(+1 -1)

>

> </details>

>

> Código e documentos foram atualizados. Vou rodar a validação completa agora; ela costuma demorar um pouco porque também passa pela validação do Firestore com Java/emulador.

>

> A validação completa passou: TypeScript, regras do Firestore, emulador com Java 21 e build de produção. Vou marcar isso no registro final para a auditoria ficar fechada com evidência.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+1 -1)

>

> </details>

>

> Vou fazer uma checagem final dos arquivos tocados para te devolver um resumo preciso, com os caminhos principais e o que ainda fica como fase seguinte.

>

> A validação de build já garante que o código compila. Vou só confirmar o modo de execução local e, se necessário, deixar um servidor local pronto para teste visual/manual.

>

> Já existe um servidor ouvindo na porta 3000, então não vou abrir outro em paralelo. Vou apenas conferir se ele responde depois das mudanças.

>

> Como houve mudança em tela, vou fazer uma verificação rápida no navegador interno também, além do build. Só preciso carregar a orientação do plugin de navegador antes de abrir a aplicação.

>

> Vou abrir a aplicação no navegador interno em segundo plano e verificar apenas se a tela carrega corretamente. Não vou mexer em dados reais nem enviar formulários.

>

> MCP tool call

>

> node_repl.js

>

> ```json

> {

> "code": "const { setupBrowserRuntime } = await
import('file:///C:/Users/marco/.codex/plugins/cache/openai-
bundled/browser/26.609.30741/scripts/browser-client.mjs');\nawait
setupBrowserRuntime({ globals: globalThis });\nglobalThis.browser = await
agent.browsers.get('iab');\nnodeRepl.write(await browser.documentation());",

> "timeout_ms": 30000,

> "title": "Conectar ao navegador"

> }

> ```

>

> ```text

> failed to write kernel assets: O sistema não pode encontrar o caminho
especificado. (os error 3)

> ```

>

> A tentativa de usar o navegador interno encontrou um problema do próprio
ambiente do plugin, então não vou insistir nisso. Como já existe servidor na porta
3000 possivelmente de uma execução anterior, vou subir uma instância nova em
outra porta para testar o código recém-buildado sem interferir no que já está
aberto.

>

> Vou usar uma porta separada e o build de produção recém-gerado para
confirmar que o servidor novo sobe limpo. Isso evita confundir validação com
algum processo antigo na porta padrão.

>

> O script `start` do projeto ainda usa sintaxe de ambiente no estilo Unix, então no Windows ele não sobe diretamente. Vou contornar isso sem alterar o script agora: iniciar o mesmo servidor com as variáveis definidas no próprio processo do Windows.

>

> O servidor novo subiu corretamente na porta 4177. Vou verificar também a página principal, porque isso confirma que o build está sendo servido e não só a rota de saúde do backend.

>

> Como encontrei um problema real no script de produção no Windows, vou corrigir isso também antes de fechar. A plataforma precisa conseguir subir pelo comando padrão no ambiente em que vocês estão usando.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/package.json` (+1 -1)

>

> </details>

>

> Vou testar o comando `start` corrigido em outra porta, para garantir que o atalho padrão agora funciona no Windows.

>

> O `start` corrigido subiu limpo. Vou rodar a validação completa uma última vez depois desse ajuste para fechar sem ressalva técnica.

>

> Validação completa passou de novo. Vou acrescentar no registro que o comando de produção também foi ajustado para Windows, porque foi um problema prático encontrado no fechamento.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+1 -0)

>

> </details>

>

> Subi duas portas durante o teste; vou limpar a primeira tentativa e deixar só uma instância nova funcionando para você testar, além da porta 3000 que já estava aberta antes.

>

> Vou encerrar apenas a instância temporária da porta 4177; a porta 4178 fica como servidor novo da entrega atual.

>

> A limpeza ficou certa: só a instância atual na porta 4178 permanece para teste. Vou fazer uma última consulta rápida nela.

>

> Fechando a checagem: build validado, servidor novo respondendo, e documentação da entrega atualizada. Vou só coletar as referências finais de linha para te entregar um resumo limpo.

>

> Uma consulta de referências falhou por aspas no comando, nada no projeto. Vou repetir de forma mais simples só para pegar as linhas finais.

</details>

Fechei esta rodada de finalização.

Implementei a inscrição de eventos pelo backend em [eventRoutes.ts](./Plataforma-Coroado/src/server/routes/eventRoutes.ts:16), conectei a tela de eventos a esse fluxo em [EventsView.tsx](./Plataforma-Coroado/src/components/EventsView.tsx:233), e registrei a rota no servidor em [server.ts](./Plataforma-Coroado/server.ts:28).

Também finalizei a conciliação financeira administrativa: a rota fica em [financeRoutes.ts](./Plataforma-Coroado/src/server/routes/financeRoutes.ts:50), a tela agora confirma/rejeita transações pendentes em [AdminFinance.tsx](./Plataforma-Coroado/src/components/admin/AdminFinance.tsx:114), e o tipo de transação agora aceita os novos estados em [index.ts](./Plataforma-Coroado/src/types/index.ts:146).

Ajustei o `npm run start` para funcionar no Windows em [package.json](./Plataforma-Coroado/package.json:15), atualizei o registro da entrega em [IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md:1) e marquei o estado final do plano em [PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md](./Plataforma-Coroado/PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md:1).

Validação final: `npm run validate` passou, incluindo TypeScript, regras Firestore, Java 21/Firestore Emulator e build de produção. Também deixei uma instância nova rodando para teste em http://localhost:4178. A porta 3000 já estava ocupada antes, então preservei.

O que ainda falta como próxima fase real, não como pendência escondida: migrar gradualmente para Next.js/tRPC, mover check-in/progresso/gestão de planos para BFF, desenhar o modelo SQL/BI para financeiro e indicadores, endurecer conciliação via webhook/banco, e depois fazer a camada visual premium com motions e testes de fluxo.

> Vamos realizar essa implementação agora:

> migrar gradualmente para Next.js/tRPC, mover check-in/progresso/gestão de planos para BFF, desenhar o modelo SQL/BI para financeiro e indicadores, endurecer conciliação via webhook/banco, e depois fazer a camada visual premium com motions e testes de fluxo.

<details><summary>75 previous messages</summary>

> Vou tratar isso como uma nova etapa grande, não como um retoque: primeiro vou mapear o estado atual do projeto e depois avançar por uma entrega vertical que já deixe Next/tRPC preparado, mais BFF para fluxos críticos e base SQL/BI documentada/validável.

>

> Vou montar um plano de execução bem pragmático: não vou tentar reescrever a plataforma inteira em Next de uma vez, porque isso quebraria o ciclo de validação. Vou criar a ponte real para tRPC/Next e, nesta mesma passada, mover mais fluxos sensíveis para backend.

>

> Vou começar pelos pontos de regra de negócio que ainda ficam mais sensíveis: check-in de evento, progresso da Escola, planos financeiros e webhook. A ideia é puxar essas decisões para o servidor, mantendo a tela simples para o usuário.

>

> O mapa confirmou três pontos bons para atacar agora: check-in ainda atualiza direto no cliente, progresso de aula ainda passa por serviço client-side, e criação de planos ainda grava direto no Firestore. Vou mover esses três para rotas autenticadas e deixar o tRPC como a nova espinha dorsal gradual.

>

> Vou adicionar o mínimo necessário para a migração gradual: Next para poder iniciar uma rota paralela da aplicação e tRPC no backend Express atual. Assim a plataforma não para esperando uma reescrita completa.

>

> Enquanto as dependências entram, vou estruturar tRPC do jeito que conversa com o backend atual: ele usa a mesma autenticação Firebase e os mesmos papéis, então não vira uma API paralela frágil.

>

> O foco da implementação agora será vertical: cada fluxo terá endpoint de backend, validação básica, controle de tenant/papel e conexão na tela. O tRPC entra como camada paralela de migração, mas as telas continuam usando chamadas simples para não forçar uma troca grande de uma vez.

>

> Também apareceu um problema de qualidade que vale corrigir junto: o fluxo de progresso usa estado otimista visual, mas grava diretamente no Firestore. Vou trocar por uma chamada autenticada que recalcula progresso no backend com base nas aulas reais do curso.

>

> Vou fazer a base Next de forma reversível: um App Router que embrulha o app atual como client component. Isso já permite rodar/migrar rota por rota depois, sem abandonar o Vite antes da hora.

>

> Agora vou aplicar os arquivos de infraestrutura: scripts Next, App Router inicial, contexto compartilhado de autenticação e rota tRPC em `/api/trpc`. Depois conecto os fluxos funcionais.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/package.json` (+7 -5)

>

> </details>

>

> Vou criar a casca Next agora. Ela não substitui o app atual; funciona como trilho paralelo para a migração gradual.

>

> <details><summary>Created 4 files</summary>

>

> - Wrote `./Plataforma-Coroado/next.config.mjs` (+6 -0)

> - Wrote `./Plataforma-Coroado/src/app/NextClientApp.tsx` (+11 -0)

> - Wrote `./Plataforma-Coroado/src/app/layout.tsx` (+19 -0)

> - Wrote `./Plataforma-Coroado/src/app/page.tsx` (+5 -0)

>

> </details>

>

> A casca Next está no lugar. Agora vou reaproveitar a mesma autenticação Firebase do Express para tRPC, evitando duas regras de permissão diferentes dentro da plataforma.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/context.ts` (+26 -14)

>

> </details>

>

> Com a autenticação centralizada, vou ampliar os contratos de dados para as novas operações: check-in, progresso de aula e criação de planos.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+31 -0)

>

> </details>

>

> Vou criar uma camada pequena de operações server-side compartilhadas. Ela evita duplicar regra entre REST e tRPC, e será o ponto natural para evoluir depois para routers tRPC mais completos.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/operations.ts` (+230 -0)

>

> </details>

>

> A operação de webhook agora vai registrar eventos de pagamento, então preciso adicionar essa coleção ao mapa central para não deixar nome solto no código.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/collections.ts` (+1 -0)

>

> </details>

>

> Vou expor essas operações em tRPC. Mesmo que o frontend ainda use chamadas REST simples, o contrato tipado já fica vivo e pronto para a migração por telas.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/trpc.ts` (+99 -0)

>

> </details>

>

> Vou registrar o endpoint tRPC no servidor Express atual, antes do fallback visual do app. Assim ele já fica disponível em `/api/trpc`.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/server.ts` (+6 -0)

>

> </details>

>

> Agora vou conectar as rotas REST atuais às operações server-side. Isso mantém o frontend simples e, ao mesmo tempo, garante que a regra já esteja do lado certo.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/eventRoutes.ts` (+27 -1)

>

> </details>

>

> Check-in saiu da atualização direta: agora a confirmação passa por backend, com validação de papel, tenant e status do pagamento. Vou fazer o mesmo para progresso da Escola.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/schoolRoutes.ts` (+27 -1)

>

> </details>

>

> Vou mover a criação de planos e reaproveitar a operação de conciliação já criada, para que o financeiro não tenha regras administrativas soltas em componentes.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/financeRoutes.ts` (+34 -24)

>

> </details>

>

> Agora vou ligar a tela financeira ao endpoint de planos. A leitura em tempo real continua pelo Firestore, mas a criação administrativa passa pelo backend.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/admin/AdminFinance.tsx` (+10 -7)

>

> </details>

>

> Vou trocar as duas atualizações de check-in que ainda eram diretas: confirmação no scanner e sincronização da fila offline.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+28 -23)

>

> </details>

>

> O `updateDoc` deixou de ser necessário na tela de eventos depois dessa mudança, então vou limpar o import para manter o código enxuto.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+1 -1)

>

> </details>

>

> Vou conectar o progresso da Escola ao backend. O servidor passa a calcular progresso usando o conjunto real de aulas do curso, em vez de aceitar uma porcentagem pronta do cliente.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+19 -11)

>

> </details>

>

> Agora vou endurecer o webhook: além de atualizar pedidos, ele também reconhecerá inscrições de eventos, registrará evento de pagamento idempotente e alimentará transações financeiras para indicadores.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/mercadoPagoRoutes.ts` (+114 -15)

>

> </details>

>

> Vou ajustar um detalhe importante: o Mercado Pago devolve vários estados, mas a tela de ingresso entende aprovado, pendente ou rejeitado. Vou normalizar isso antes de gravar na inscrição.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/mercadoPagoRoutes.ts` (+7 -1)

>

> </details>

>

> Vou incluir `payment\_events` nas regras do Firestore como coleção somente administrativa. Mesmo sendo escrita pelo Admin SDK, isso evita uma superfície aberta se alguém tentar acessar pelo cliente.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/firestore.rules` (+5 -0)

>

> </details>

>

> Agora vou colocar duas bases que dão continuidade à auditoria: um modelo SQL/BI versionado para Cloud SQL/PostgreSQL e um smoke test de fluxos para validar endpoints sem tocar em dados reais.

>

> <details><summary>Created 3 files</summary>

>

> - Wrote `./Plataforma-Coroado/docs/sql-bi/README.md` (+29 -0)

> - Wrote `./Plataforma-Coroado/docs/sql-bi/coroado\_finance\_indicators\_model.sql` (+198 -0)

> - Wrote `./Plataforma-Coroado/scripts/flow-smoke-tests.mjs` (+39 -0)

>

> </details>

>

> Vou incluir o smoke test nos scripts do projeto, separado da validação pesada para não depender sempre de um servidor local aberto.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/package.json` (+1 -0)

>

> </details>

>

> Agora vou adicionar uma camada visual inicial reaproveitável: presets de motion e aplicação no financeiro administrativo, que é uma das telas mais operacionais e sensíveis.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/motion/presets.ts` (+19 -0)

>

> </details>

>

> Vou aplicar esses presets na tela financeira sem mudar a estrutura visual: entrada suave da página e listas com movimento leve, próprio de ferramenta operacional.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/admin/AdminFinance.tsx` (+18 -8)

>

> </details>

>

> A implementação funcional principal está montada. Vou rodar a validação agora para encontrar incompatibilidades de TypeScript/Next/tRPC antes de avançar para documentação final e testes de fluxo.

>

> A primeira checagem pegou só um ajuste simples na casca Next: o layout precisava importar explicitamente o tipo de nó React. Vou corrigir e repetir.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/layout.tsx` (+2 -1)

>

> </details>

>

> TypeScript passou. Vou validar também o build do Next agora, porque a migração gradual só conta de verdade se a casca paralela compilar.

>

> O Next build encontrou uma diferença real entre Vite e Next: dois imports globais de CSS que o Vite aceita não são resolvidos do mesmo jeito pelo Next/Turbopack. Vou criar uma folha CSS própria para a casca Next, sem mexer no CSS principal da SPA.

>

> Vou dar ao Next uma folha CSS própria, compatível com o build dele, mantendo o CSS original intacto para o Vite. Isso deixa a migração paralela viável sem desmontar o app atual.

>

> <details><summary>Created a file, edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/layout.tsx` (+1 -1)

> - Wrote `./Plataforma-Coroado/src/app/next.css` (+81 -0)

>

> </details>

>

> Ajustei a camada CSS do Next. Vou rodar o build Next outra vez para validar a ponte de migração.

>

> O Next build avançou bem: compilou, instalou o tipo React que faltava, e agora expôs um problema real de tipagem que o Vite não barrava: um botão usando `asChild` sem suporte no componente atual. Vou corrigir essa incompatibilidade no app.

>

> Vou trocar esse botão por um clique direto de navegação. É uma pequena correção, mas boa: remove uma prop que o componente não implementa.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/App.tsx` (+6 -6)

>

> </details>

>

> Vou repetir o build Next; agora ele deve continuar a revelar apenas incompatibilidades reais do app com a futura migração.

>

> O Next segue fazendo uma boa auditoria de tipagem. Agora ele encontrou um `useRef` sem valor inicial em uma tela de jogo/jornada; vou ajustar sem mudar comportamento.

>

> Vou ajustar o ref da animação para aceitar `null`, que é a forma correta com os tipos atuais do React.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/components/JornadaView.tsx` (+1 -1)

>

> </details>

>

> Vou rodar o build Next novamente. Esses ajustes vão deixando o código mais preparado para a migração real, não só para o empacotador atual.

>

> Mais uma incompatibilidade útil: dados de perfil vindos do Firestore chegam como tipo amplo. Vou explicitar conversão para string no avatar, sem alterar a UI.

>

> Vou deixar o avatar do layout mais seguro para dados vindos do perfil.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/components/Layout.tsx` (+1 -1)

>

> </details>

>

> Vou repetir o build Next. Cada erro que aparece aqui é um ponto onde a plataforma fica menos dependente de tolerâncias do build antigo.

>

> A ponte Next compilou com sucesso. Vou avançar para fechar o restante: validação completa da SPA, smoke tests de fluxo e documentação do que foi implementado nesta etapa.

>

> Vou rodar a validação completa tradicional agora: TypeScript, regras Firestore, emulador com Java e build Vite.

</details>

A validação principal passou. Vou reiniciar a instância local de teste com o código novo e rodar os smoke tests de fluxo que acabei de adicionar.

> Continue

<details><summary>11 previous messages</summary>

> Continuando. A instância local já respondeu em `4178`; aquele aviso foi só o arquivo de log ainda preso por um processo antigo enquanto eu reiniciava o servidor. Vou seguir com os smoke tests e, se aparecer algo real, corrijo antes de fechar.

>

> Os smoke tests passaram: saúde do backend, tRPC, rotas protegidas exigindo autenticação e webhook vazio sendo aceito/ignorado corretamente. Vou só adicionar um atalho de validação completa incluindo Next para que essa nova frente não fique “fora do radar”.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/package.json` (+4 -3)

>

> </details>

>

> Vou atualizar os documentos de entrega com esta nova rodada: ponte Next/tRPC, BFF ampliado, webhook mais forte, SQL/BI e camada visual. Depois rodo uma última checagem rápida.

>

> <details><summary>Edited 2 files</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+11 -0)

> - Wrote `./Plataforma-Coroado/PLANO\_EVOLUCAO\_STACK\_AUDITORIA\_COROADO\_2026-06-14.md` (+6 -0)

>

> </details>

>

> A documentação já reflete a nova entrega. Vou rodar o novo atalho `validate:full` agora; ele combina a validação tradicional com o build Next, justamente para garantir que as duas frentes continuem saudáveis.

>

> `validate:full` passou: Vite, Firestore/Java, regras, TypeScript e Next build. Vou só confirmar o servidor local e coletar os arquivos-chave para te entregar um fechamento claro do que foi feito e do que fica para a próxima camada.

>

> A checagem final mostrou artefatos gerados pelo Next (`.next` e cache TypeScript). Vou limpar esses arquivos de build local e garantir que eles fiquem ignorados, para a entrega não misturar código com saída gerada.

>

> Vou adicionar as saídas do Next/TypeScript ao `.gitignore` e remover só os artefatos gerados dentro da pasta da plataforma.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/.gitignore` (+2 -0)

>

> </details>

>

> Limpeza feita. Vou fechar o plano como concluído e pegar o status final da árvore para não deixar nada ambíguo.

</details>

Avancei uma entrega grande e validada.

Implementei a ponte gradual para Next.js em [src/app](./Plataforma-Coroado/src/app/page.tsx:1), mantendo a SPA atual funcionando. Também adicionei tRPC em [/api/trpc](./Plataforma-Coroado/server.ts:35), com operações compartilhadas em [operations.ts](./Plataforma-Coroado/src/server/operations.ts:1).

Movi mais fluxos críticos para o BFF:

- Check-in de eventos: [eventRoutes.ts](./Plataforma-Coroado/src/server/routes/eventRoutes.ts:17) e tela em [EventsView.tsx](./Plataforma-Coroado/src/components/EventsView.tsx:336)
- Progresso da Escola IDE: [schoolRoutes.ts](./Plataforma-Coroado/src/server/routes/schoolRoutes.ts:14) e tela em [SchoolView.tsx](./Plataforma-Coroado/src/components/SchoolView.tsx:2208)
- Gestão de planos: [financeRoutes.ts](./Plataforma-Coroado/src/server/routes/financeRoutes.ts:16) e tela em [AdminFinance.tsx](./Plataforma-Coroado/src/components/admin/AdminFinance.tsx:60)

Também endureci o webhook Mercado Pago para alimentar pedidos, inscrições de eventos, transações e `payment\_events`; criei o modelo SQL/BI em [coroado_finance_indicators_model.sql](./Plataforma-Coroado/docs/sql-bi/coroado_finance_indicators_model.sql:1); e iniciei a camada visual premium com presets em [presets.ts](./Plataforma-Coroado/src/lib/motion/presets.ts:1).

Validações passaram:

- `npm run validate:full`
- `npm run test:flows`
- servidor local respondendo em http://localhost:4178

O que ainda fica para a próxima etapa: migrar telas rota por rota para Next de verdade, criar o worker de sincronização Firestore -> SQL/BI, integrar conciliação bancária real conforme provedor escolhido, expandir motions para as telas principais e adicionar testes e2e autenticados. Também apareceu alerta de auditoria do `npm install` com vulnerabilidades em dependências; não rodei `npm audit fix --force` porque pode quebrar versões e precisa ser uma entrega própria.

> Avance essa próxima etapa de migrar as telas rota por rota, e criar worker de sincronização. Integração com o mercado pago (nao sei se possui SDK proprio) precisamos tanto pra vendas pontuais, de produtos, eventos ou cursos pontuais como tbm precisamos de assinatura(para assinar a escola IDE). E a expansao motion para as telas

<details><summary>129 previous messages</summary>

> Vou avançar por uma entrega incremental de verdade: criar rotas Next explícitas para as áreas principais sem quebrar o Vite atual, adicionar worker de sincronização Firestore -> SQL/BI, ampliar Mercado Pago para compra pontual e assinatura, e espalhar motion em um conjunto maior de telas operacionais. Primeiro vou mapear as rotas e os pontos de pagamento atuais para entrar sem rasgar a base.

>

> <details><summary>Searched the web</summary>

>

> <details><summary>Searched the web</summary>

>

> - Searched the web for `Mercado Pago Node.js SDK PreApproval Preference official documentation`

>

> </details>

>

> </details>

>

> Confirmei pela documentação oficial que o Mercado Pago tem SDK server-side para Node.js e API própria de assinaturas via `preapproval`; o pacote `mercadopago` já está instalado no projeto. Vou implementar usando o SDK onde ele já encaixa bem e manter fallback por API HTTP para webhook/consulta, que já estava funcionando.

>

> Vou começar pela migração de rotas: em vez de uma única página Next na raiz, cada área principal ganhará sua própria rota no App Router. Elas ainda renderizam o app atual por trás, mas já viram pontos de migração independentes.

>

> Vou adicionar as páginas Next explícitas para cada rota de área. Isso ainda usa o wrapper atual, mas cria o mapa real do App Router para `/escola`, `/eventos`, `/loja`, `/gestao` etc., que é o começo seguro da migração tela por tela.

>

> <details><summary>Created 14 files</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/celulas/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/cuidado-pastoral/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/escola/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/eventos/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/financeiro/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/gestao/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/jornada/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/loja/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/membros/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/midia/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/ministerios/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/pastores/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/social/page.tsx` (+5 -0)

> - Wrote `./Plataforma-Coroado/src/app/unidades/page.tsx` (+5 -0)

>

> </details>

>

> Agora vou criar o worker de sincronização Firestore -> SQL/BI. Ele será operacional, mas seguro: só roda quando houver `DATABASE\_URL`, e faz upsert nas tabelas do modelo SQL já criado.

>

> O PowerShell daqui não aceita `&&` como separador. Vou instalar os pacotes em dois passos.

>

> Vou criar o worker com três modos úteis: aplicar o schema SQL, sincronizar dados, e rodar em modo `--dry-run` para conferir configuração sem gravar. Isso ajuda a usar em Cloud Run, scheduler ou execução manual depois.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/scripts/sync-firestore-to-sql.ts` (+300 -0)

>

> </details>

>

> O worker foi criado. Vou adicioná-lo aos scripts do projeto para rodar localmente, em Cloud Run ou em uma rotina agendada.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/package.json` (+5 -3)

>

> </details>

>

> Vou avançar Mercado Pago agora: compras pontuais já existem para loja/eventos, então vou criar uma camada unificada que também aceite curso/aula avulsa e assinatura da Escola IDE.

>

> Vou implementar assinatura da Escola IDE e compra avulsa de curso/aula como fluxos separados: assinatura via `preapproval` do Mercado Pago, e compra avulsa via preferência de checkout normal, ambas reconciliadas pelo webhook.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/collections.ts` (+2 -0)

>

> </details>

>

> Vou adicionar os contratos de assinatura e compra escolar para o backend
validar os payloads antes de falar com Mercado Pago.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/domain/platform-contracts.ts` (+12 -0)

>

> </details>

>

> Agora vou criar as rotas server-side da Escola para assinatura e compra avulsa.
Elas vão usar o SDK Node do Mercado Pago já instalado.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/schoolRoutes.ts` (+205 -2)

>

> </details>

>

> As rotas da Escola agora precisam conhecer a origem da aplicação para montar
o retorno do Mercado Pago. Vou passar a porta/origem pelo bootstrap do servidor.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/server.ts` (+1 -1)

>

> </details>

>

> Vou completar a reconciliação: quando o webhook aprovar compra avulsa, ele libera acesso ao curso/aula; quando aprovar assinatura, ele ativa a Escola IDE no perfil do usuário e no histórico de assinaturas.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/mercadoPagoRoutes.ts` (+95 - 0)

>

> </details>

>

> Vou ajustar o webhook para aceitar tanto notificações de pagamento quanto notificações de assinatura no mesmo endpoint principal, porque o Mercado Pago pode enviar `preapproval` / `subscription\_preapproval` conforme configuração.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/mercadoPagoRoutes.ts` (+64 - 53)

>

> </details>

>

> Vou proteger a nova coleção de acesso comprado: usuário lê o próprio acesso, backend escreve via Admin SDK, cliente não grava direto.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/firestore.rules` (+5 -0)

>

> </details>

>

> Vou conectar a tela da Escola aos novos endpoints: assinatura passa pelo BFF, e compra avulsa de aula ganha checkout real em vez de alerta. Também vou considerar acessos já comprados para destravar aulas.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+20 -21)

>

> </details>

>

> Agora vou adicionar leitura de acessos comprados e ação de compra avulsa no player da Escola.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+60 -6)

>

> </details>

>

> Vou substituir o QR Pix visual de teste por uma ação real de checkout Mercado Pago e estado de carregamento.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+12 -14)

>

> </details>

>

> Vou remover o import de QR que ficou sobrando na Escola depois da troca para checkout real.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/components/SchoolView.tsx` (+0 -1)

>

> </details>

>

> Vou também adicionar compra avulsa de curso inteiro no detalhe do curso, além da compra de aula dentro do player.

>

> Vou colocar também o botão de compra do curso inteiro no painel lateral do curso, quando houver preço configurado.

>

> Vou aplicar o patch usando os trechos exatos do arquivo.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/components/SchoolView.tsx` (+32 -6)

>

> </details>

>

> Vou inserir o botão logo antes de “Adicionar à Lista”.

>

> O trecho tem caracteres herdados de codificação em volta, então vou usar uma âncora menor.

>

> Vou localizar a linha pelo conteúdo renderizado e aplicar com o texto real.

>

> Agora sim, vou usar a string correta com acento.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+11 -1)

>

> </details>

>

> Vou corrigir o rótulo de preço do curso para não dizer “Gratuito” quando houver preço avulso configurado.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+5 -1)

>

> </details>

>

> Vou fazer o detalhe do curso também respeitar acessos já comprados, para que uma aula/curso comprado não continue aparecendo como bloqueado.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+28 -5)

>

> </details>

>

> Vou expandir motion agora de modo controlado: aplicar entrada suave nas telas principais que ainda não usam presets, sem mexer em lógica nem criar animações chamativas.

>

> Essas telas já usam `motion`, então vou padronizar os presets nelas para que a transição fique consistente entre áreas.

>

> Vou aplicar o preset de página nessas telas, mantendo as animações internas que elas já tinham.

>

> Vou aplicar em cada arquivo com âncoras menores, para não esbarrar em blocos longos.

>

> <details><summary>Edited 4 files</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/HomeView.tsx` (+5 -4)

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+5 -4)

> - Wrote `./Plataforma-Coroado/src/components/FinanceView.tsx` (+5 -4)

> - Wrote `./Plataforma-Coroado/src/components/StoreView.tsx` (+5 -4)

>

> </details>

>

> Vou colocar motion também no painel inicial autenticado, porque usuários aprovados caem nele antes da Home pública.

>

> Vou envolver o dashboard autenticado no mesmo preset de página.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/HomeDashboard.tsx` (+4 -2)

>

> </details>

>

> Vou ampliar os smoke tests para cobrir os novos endpoints de assinatura e compra da Escola, garantindo que estejam protegidos por autenticação.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/scripts/flow-smoke-tests.mjs` (+2 -0)

>

> </details>

>

> A parte de implementação principal está pronta. Vou rodar TypeScript agora para encontrar quebras de contrato antes de validar tudo.

>

> Dois ajustes pequenos de tipagem: a resposta de assinatura do SDK não tipa `sandbox\_init\_point`, e o Zod deixou o `targetId` mais amplo do que o necessário no TypeScript. Vou corrigir sem mudar comportamento.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/server/routes/schoolRoutes.ts` (+12 -8)

>

> </details>

>

> Vou repetir a checagem.

>

> <details><summary>Searched the web</summary>

>

> <details><summary>Searched the web</summary>

>

> - Searched the web for `https://github.com/mercadopago/sdk-nodejs`

>

> </details>

>

> </details>

>

> Como agora assinaturas e acessos comprados entram no produto, vou estender também o modelo SQL/BI e o worker para esses dois fatos.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/docs/sql-bi/coroado\_finance\_indicators\_model.sql` (+38 -0)

>

> </details>

>

> Context compaction

>

> Source: automatic

> Status: completed

>

> Vou continuar dessa base: fechar o worker de sincronização com assinaturas/acessos, amarrar a compra avulsa e recorrente no modelo de dados, depois rodar as validações para ver onde o app reclama de verdade.

>

> Estou olhando agora os arquivos que mudaram no trecho de escola, Mercado Pago e sincronização. A ideia é corrigir já no ponto de integração, porque esses

três pedaços precisam falar a mesma língua: pagamento, liberação de acesso e indicador futuro.

>

> O modelo SQL já ganhou as tabelas novas, mas o worker ainda não estava levando `subscriptions` e `learning\_access` para lá. Vou completar essa ponte agora e atualizar a documentação curta do BI para que a próxima pessoa entenda o que entra em cada indicador.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/scripts/sync-firestore-to-sql.ts` (+35 -0)

>

> </details>

>

> A sincronização agora cobre também assinaturas e acessos comprados. Isso fecha um ciclo importante: o pagamento cria/libera acesso no operacional e o mesmo fato passa a existir na camada de indicadores.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/docs/sql-bi/README.md` (+4 -1)

>

> </details>

>

> Agora vou rodar as validações locais para deixar o compilador e as regras do Firebase apontarem o que ainda está frouxo. Se aparecer erro, sigo direto para correção.

>

> Boa: tipagem passou e o worker já responde em modo seco quando ainda não há banco SQL conectado. Vou fazer a validação completa agora, que costuma pegar problemas de build, regras e integração entre pacotes.

>

> A validação completa passou, incluindo Java/Firestore, Vite e Next com as rotas novas geradas. Vou agora testar os fluxos HTTP protegidos para garantir que as rotas novas não ficaram só “bonitas no build”.

>

> O build está certo, mas o teste de fluxo bateu em um servidor antigo/sem a rota nova e recebeu 404. Vou reiniciar a instância local da plataforma e rodar o teste de novo contra o backend atualizado.

>

> Para não derrubar nada que talvez você esteja usando em paralelo, vou subir uma instância nova em outra porta e apontar os testes para ela. Assim validamos o código atual sem mexer no que já estava aberto.

>

> A instância nova não respondeu imediatamente, então vou checar o log da porta nova. Às vezes o servidor sobe alguns segundos depois porque o Vite prepara o middleware primeiro.

>

> Servidor novo está saudável em `4180`; só há um aviso do websocket de recarregamento do Vite usando uma porta já ocupada, sem bloquear o backend. Vou rodar os fluxos agora contra essa instância atualizada.

>

> Os fluxos protegidos passaram na instância atualizada. Vou fazer uma checagem visual rápida da aplicação local para confirmar que as telas migradas continuam abrindo depois dos wrappers de motion e das rotas novas.

>

> Vou conectar no navegador interno do Codex para abrir a plataforma local em segundo plano. Se essa ponte do navegador falhar, sigo com as validações automatizadas que já passaram e deixo isso registrado com clareza.

>

> MCP tool call

>

> node_repl.js

```
>
> ```json
> {
>   "code": "const { setupBrowserRuntime } = await
import('file:///C:/Users/marco/.codex/plugins/cache/openai-
bundled/browser/26.609.30741/scripts/browser-client.mjs');\nawait
setupBrowserRuntime({ globals: globalThis });\nglobalThis.browser = await
agent.browsers.get('iab');\nnodeRepl.write(await browser.documentation());",
>   "timeout_ms": 30000,
>   "title": "Conectar navegador interno"
> }
> ```
>
> ```text
> failed to write kernel assets: O sistema não pode encontrar o caminho
especificado. (os error 3)
> ```
>
> O navegador interno falhou na conexão por um erro do próprio ambiente local ao
preparar os arquivos de apoio. Como a aplicação já passou no build completo e
nos fluxos HTTP, vou continuar com validações de rota/HTML pelo servidor local e
registrar essa limitação no fechamento.
>
> Vou checar rapidamente os pontos de pagamento já existentes para produtos e
eventos, porque a nova parte da Escola IDE deve complementar esse trilha, não
criar um segundo padrão escondido.
>
> Achei um ponto importante: produtos já usam checkout no backend, e eventos
já criam preferência, mas a tela de eventos ainda carrega um Brick com chave
pública de teste. Vou simplificar para o mesmo padrão robusto: backend cria a
preferência e a tela redireciona para o checkout oficial do Mercado Pago.
>
```

> Vou remover esse ponto de fragilidade da tela de eventos: nada de chave pública falsa no front. O ticket pago vai guardar o link seguro retornado pelo backend e permitir retomar o pagamento por ali.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/eventRoutes.ts` (+1 -0)

>

> </details>

>

> O backend de eventos agora guarda o link do checkout junto do ingresso pendente. Vou trocar a área “Meus Ingressos” para usar esse link, sem depender de componente de pagamento inicializado no navegador.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+7 -10)

>

> </details>

>

> Agora vou substituir o bloco visual antigo do pagamento por um estado simples e acionável: pendente, botão para continuar o checkout, e QR code só quando o ingresso já estiver liberado.

>

> O trecho tem pequenas diferenças de espaçamento/encoding, então vou localizar o bloco exato antes de aplicar a troca.

>

> Encontrei o bloco. Vou aplicar a substituição por faixas menores para evitar que a codificação antiga do arquivo atrapalhe a edição.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+1 -1)

>

> </details>

>

> A moldura do ingresso agora já diferencia visualmente “pendente” de “liberado”. Vou trocar o conteúdo interno antigo pelo estado novo com botão de retomada.

>

> A edição falhou de novo por alguma diferença invisível no trecho antigo. Vou ler esse pedaço em formato literal para substituir com segurança sem mexer no restante da tela.

>

> MCP tool call

>

> node_repl.js

>

> ```json

> {

> "code": "const fs = await import('node:fs');\nconst text = fs.readFileSync('D:/Projetos/Coroado/Plataforma-Coroado/src/components/EventsView.tsx','utf8');\nconst start = text.indexOf('<div className=\\\"flex flex-col items-center justify-center text-center w-full max-w-[400px]\\\">');\nconst end = text.indexOf(') : (',\nstart);\nnodeRepl.write(JSON.stringify({start,end,snippet:text.slice(start,end)},\nnull, 2));",

> "timeout_ms": 30000,

> "title": "Ler trecho literal"

> }

> ```

>

> ```text

> failed to write kernel assets: O sistema não pode encontrar o caminho especificado. (os error 3)

> ```

>

> A sessão de navegador deixou a execução JavaScript interna indisponível também, então vou usar uma leitura local simples para inspecionar só o texto. Continuo sem alterar arquivo por esse caminho.

>

> A primeira inspeção não encontrou a string por conta das aspas/escapes. Vou buscar pelo componente antigo diretamente.

>

> Agora o trecho apareceu com a codificação correta. Vou aplicar a substituição usando exatamente esse conteúdo localizado.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/EventsView.tsx` (+18 -24)

>

> </details>

>

> Com isso, eventos pagos também seguem o fluxo de checkout hospedado. Vou procurar se sobrou algum uso do SDK React antigo antes de decidir se ele pode sair das dependências.

>

> Não há mais uso real do SDK React do Mercado Pago. Como o checkout agora nasce no backend, vou remover essa dependência para reduzir superfície de erro no front.

>

> Dependência antiga removida. Vou rodar novamente a tipagem para garantir que não sobrou referência invisível.

>

> Vou reforçar os testes de fluxo para cobrir também os pontos pagos que ficaram no padrão novo: checkout da loja e inscrição paga de eventos devem exigir autenticação antes de qualquer coisa.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/scripts/flow-smoke-tests.mjs` (+2 -0)

>

> </details>

>

> Testes reforçados. Vou rodar a validação completa de novo, porque mexemos em dependência, evento e backend.

>

> Validação completa passou de novo. Como o servidor local foi iniciado antes da última alteração em eventos, vou subir outra instância limpa e rodar os testes de fluxo reforçados nela.

>

> Vou confirmar se a nova porta respondeu antes de executar o teste.

>

> A nova instância está saudável. Vou rodar os fluxos protegidos e o webhook vazio contra ela.

>

> Fluxos passaram com os novos pontos de pagamento cobertos. Vou atualizar os documentos da entrega para deixar registrado o que entrou nesta etapa e o que ainda falta para fechar a migração rota por rota.

>

> Vou abrir os dois registros principais da auditoria/evolução e adicionar esta etapa como bloco de implementação concluída, sem reescrever o histórico anterior.

>

> Os documentos estão com histórico em uma codificação antiga, então vou anexar um bloco novo em texto simples/ASCII para não corromper o que já existe.

>

> Vou localizar a última linha real do documento para anexar o bloco no ponto certo.

>

> Achei o fim do arquivo. Vou anexar o bloco depois da recomendação SQL.

>

> O `apply\_patch` não está conseguindo casar essa última linha por causa da codificação. Vou anexar usando o marcador de fim de arquivo do próprio patch.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+26 -0)

>

> </details>

>

> O bloco entrou, mas pelo comportamento do patch foi inserido no início do documento. Vou ajustar para ficar no fim como registro de etapa, mantendo o histórico legível.

>

> O bloco ficou no ponto correto, depois da seção SQL. Vou registrar o mesmo avanço no plano das 6 fases, em formato de status.

>

> Vou anexar o status pós-implementação ao fim do plano para manter claro o que mudou de fase.

>

> O fim desse arquivo também tem codificação antiga. Vou anexar o bloco pelo patch de fim de arquivo, como no documento anterior.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-

Coroado/PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md`

(+32 -0)

>

> </details>

>

> Vou checar onde o bloco entrou para confirmar que o plano ficou organizado.

>

> Os registros ficaram no fim dos documentos corretos. Vou fazer uma última revisão de status dos arquivos e confirmar que nada óbvio ficou pendente de validação.

>

> Não sobrou nenhum uso do SDK React antigo do Mercado Pago. Vou confirmar só a dependência do `pg`, porque o worker SQL precisa dela em runtime.

>

> Achei uma inconsistência: o tipo do `pg` entrou, mas a dependência runtime `pg` não ficou registrada diretamente no `package.json`. Vou corrigir isso agora, porque o worker depende dela quando rodar fora da máquina local.

>

> Agora o worker tem a dependência runtime declarada. Vou rodar as checagens finais mais leves de tipagem, dry-run do worker e fluxo HTTP.

>

> Essas checagens passaram. Como houve ajuste de dependência depois do build completo, vou rodar a validação completa uma última vez para fechar sem ponta solta.

>

> Fechou: validação completa passou outra vez. Vou marcar a etapa como concluída e preparar o resumo do que foi implementado e o que ainda falta.

>

> Vou encerrar as instâncias locais de teste que subi nas portas `4180` e `4181`, para não deixar processos ocupando porta sem necessidade.

</details>

Avancei a próxima etapa e deixei a base mais madura para a migração gradual.

Implementei rotas explícitas no Next para as principais áreas, criei o worker de sincronização Firestore -> SQL/BI, expandi o modelo analítico para assinaturas/acessos da Escola IDE, consolidei Mercado Pago no backend para produtos, eventos, cursos/aulas avulsas e assinatura recorrente, e apliquei a primeira camada de motion nas telas principais.

Arquivos centrais desta etapa:

- [scripts/sync-firestore-to-sql.ts](./Plataforma-Coroado/scripts/sync-firestore-to-sql.ts)
- [src/server/routes/schoolRoutes.ts](./Plataforma-Coroado/src/server/routes/schoolRoutes.ts)
- [src/server/routes/mercadoPagoRoutes.ts](./Plataforma-Coroado/src/server/routes/mercadoPagoRoutes.ts)
- [src/server/routes/eventRoutes.ts](./Plataforma-Coroado/src/server/routes/eventRoutes.ts)
- [src/components/SchoolView.tsx](./Plataforma-Coroado/src/components/SchoolView.tsx)
- [src/components/EventsView.tsx](./Plataforma-Coroado/src/components/EventsView.tsx)
- [docs/sql-bi/coroado_finance_indicators_model.sql](./Plataforma-Coroado/docs/sql-bi/coroado_finance_indicators_model.sql)

Sobre Mercado Pago: sim, existe SDK oficial para Node.js. Usei o padrão backend com `mercadopago`, `Preference` para compras pontuais e `PreApproval` para assinatura da Escola IDE. Fontes oficiais: [SDK Node.js](https://github.com/mercadopago/sdk-nodejs), [SDKs server-side](https://www.mercadopago.com.br/developers/en/docs/sdks-library/server-

side),

[PreApproval/subscriptions](https://www.mercadopago.com.co/developers/en/reference/online-payments/subscriptions/create-preapproval/post).

Validações passaram:

- `npm run validate:full`
- `npm run sync:bi -- --dry-run`
- `FLOW\_BASE\_URL=http://localhost:4181 npm run test:flows`

O que ainda falta para a próxima rodada: migrar cada tela para componentes Next nativos de verdade, conectar `DATABASE\_URL` /Cloud SQL e agendar o worker, configurar credenciais/webhooks reais do Mercado Pago no ambiente, aprofundar motion por fluxo interno e atacar o relatório de `npm audit`, que ainda aponta 31 vulnerabilidades. A checagem visual pelo navegador interno falhou por limitação do ambiente do plugin, mas build, Firestore Emulator, Next e smoke tests passaram.

> Avance essa próxima fase com a migração. Não temos ainda um data base URL. Mas o firebase cliente está instalado. Pode criar e configurar o database e já incluir. Pelo firebase cli acessa o firestore tbm e todos demais pontos

<details><summary>55 previous messages</summary>

> Vou avançar com cuidado aqui porque “database URL” pode significar duas coisas no projeto: o `DATABASE\_URL` do SQL/BI que criamos para o worker, ou o `databaseURL` do Firebase client. Vou primeiro ler a configuração atual e usar o Firebase CLI para descobrir qual projeto/Firestore está ativo antes de criar ou preencher qualquer coisa.

>

> A memória confirmou que o alvo correto continua sendo `D:\Projetos\Coroador\Plataforma-Coroador`, não as variantes antigas. Agora vou separar o que é Firebase operacional do que é SQL/BI, porque se tentarmos inventar um `DATABASE\_URL` sem Cloud SQL real a plataforma ganha uma configuração falsa.

>

> Encontrei dois pontos objetivos: não existe `.firebaserc` ainda e o comando global `firebase` não está disponível. Vou usar o CLI via pacote do projeto (`firebase-tools`) se ele existir, ou instalar como ferramenta de desenvolvimento local, e também localizar o arquivo real de inicialização do Firebase client.

>

> O Firebase client já existe em `lib/firebase.ts` e usa `firebase-applet-config.json`. Vou ler essa configuração e o contexto server-side para saber se a base está apontando para um projeto real ou ainda está em modo applet/demo.

>

> Esse app está usando um Firestore com `databaseId` nomeado (`ai-studio-...`), não apenas o banco padrão. Vou preparar o CLI local e ajustar a configuração para esse `databaseId` específico, senão deploy/validação podem mirar o banco errado.

>

> A validação já usa `npx firebase-tools`, então não preciso depender de instalação global. Vou consultar o projeto com o CLI via `npx` e, se houver sessão autenticada, tentar listar os bancos Firestore do projeto real.

>

> O CLI via `npx` respondeu e encontrou o projeto `gen-lang-client-0529830528`. Vou verificar agora quais comandos de Firestore/database estão disponíveis nessa versão e listar os bancos do projeto.

>

> O banco Firestore real já existe e é exatamente o `databaseId` que o app usa: `ai-studio-534c2e7e-8664-4b76-95e3-faf31fc1628b`. Então não vou criar outro banco duplicado; vou configurar o projeto local para esse banco e preparar comandos de leitura/deploy mirando nele.

>

> <details><summary>Searched the web</summary>

>

> <details><summary>Searched the web</summary>

>

> - Searched the web for `Firebase CLI firebase.json firestore named database rules databaseId`

> - Searched the web for `https://firebase.google.com/docs/rules/manage-deploy`

> - Searched the web for `https://firebase.google.com/docs/rules/manage-deploy`

> - Searched the web for ` 'additional named databases' in https://firebase.google.com/docs/cli`

> - Searched the web for `https://firebase.google.com/docs/cli`

>

> </details>

>

> </details>

>

> Achei uma correção maior do que só `.firebase` : o client usa o banco nomeado, mas o backend Admin SDK ainda estava chamando Firestore padrão. Vou centralizar o `databases` e fazer o BFF/worker/Functions olharem para o mesmo Firestore do client.

>

> Vou editar agora as configurações: `.firebase`, `firebase.json` com `databases` nomeado, `.env.example` /scripts e o Admin SDK para usar o mesmo banco do client. Isso evita a situação perigosa de front gravar em um Firestore e backend consultar outro.

>

> Também vou incluir comandos npm para operar o Firebase sem depender de CLI global: listar bancos, ver indexes do banco nomeado e fazer dry-run/deploy seletivo de regras. O deploy real eu deixo como comando pronto, mas a validação automática usará dry-run para não sobrescrever produção sem necessidade.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroador/.firebase` (+6 -0)

>

> </details>

>

> A associação local do projeto Firebase foi criada. Agora vou fazer o
`firebase.json` mirar o Firestore nomeado que o app já usa.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/firebase.json` (+7 -4)

>

> </details>

>

> Vou alinhar o backend Admin SDK ao mesmo databaseId. Essa é a parte que
garante que as rotas BFF, webhooks e worker leiam o mesmo Firestore que as
telas estão usando.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/server/context.ts` (+4 -2)

>

> </details>

>

> Vou fazer o mesmo ajuste no código das Cloud Functions, porque elas ainda
usavam o Firestore padrão.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/functions/src/index.ts` (+13 -6)

>

> </details>

>

> Agora vou registrar as variáveis esperadas e scripts operacionais. Assim qualquer máquina consegue usar o Firebase CLI com o projeto certo sem depender de lembrar os IDs.

>

> O arquivo de exemplo tem acentuação antiga, então vou localizar as linhas exatas e aplicar uma inserção menor.

>

> Vou inserir as variáveis novas logo depois de `APP\_URL`, preservando o restante.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/.env.example` (+13 -3)

>

> </details>

>

> Agora vou adicionar scripts npm para consultar Firestore pelo CLI, validar configuração e fazer dry-run/deploy seletivo das regras do banco nomeado.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/package.json` (+8 -3)

>

> </details>

>

> Com a base Firebase alinhada, vou avançar a migração: em vez de todas as páginas Next apenas renderizarem a SPA genérica, vou passar a rota desejada para o app cliente. Assim `/escola`, `/eventos`, `/financeiro` etc. já abrem diretamente na área correta dentro do App Router, que é o próximo passo natural antes de extrair tela por tela.

>

> Para a migração rota por rota renderizar melhor, vou trocar os imports pesados das telas por carregamento sob demanda. Isso começa a transformar cada rota em um bloco carregável, sem quebrar a SPA atual nem o App Router.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/App.tsx` (+22 -21)

>

> </details>

>

> Imports pesados viraram carregamento sob demanda. Vou remover o provider de células que era só um wrapper vazio e envolver as rotas com um carregamento visual consistente.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/App.tsx` (+15 -17)

>

> </details>

>

> Vou colocar um fallback discreto nas rotas. Isso evita tela vazia quando uma área pesada está carregando sob demanda.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/App.tsx` (+32 -30)

>

> </details>

>

> Vou adicionar o componente de carregamento da rota no final do arquivo.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/App.tsx` (+18 -7)

>

> </details>

>

> Vou validar tipagem agora, porque mexemos na estratégia de carregamento das telas e no Admin SDK do Firestore.

>

> Tipagem e Functions passaram; o CLI conseguiu listar os bancos e indexes reais do Firestore nomeado. Vou fazer agora um dry-run de deploy das regras para confirmar que o `firebase.json` novo está aceito pelo Firebase.

>

> O dry-run de deploy passou, mas o Firebase avisou que uma função de regra (`isValidSubscription`) está sem uso. Vou limpar isso para deixar as regras sem ruído.

>

> Vou reaproveitar essa validação nas regras de assinatura para remover o aviso e fortalecer escrita administrativa.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/firestore.rules` (+2 -1)

>

> </details>

>

> Vou também consultar os detalhes do banco ativo e salvar um pequeno documento operacional com os comandos de Firebase/Firestore, incluindo o que depende de Cloud SQL futuro.

>

> Vou criar a documentação curta da configuração Firebase para a plataforma: projeto, banco, scripts e o limite atual do SQL/BI.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/docs/firebase/README.md` (+32 -0)

>

> </details>

>

> Com a documentação criada, vou repetir o dry-run das regras para confirmar que o aviso sumiu e que o databaseId nomeado continua aceito.

>

> O dry-run de deploy das regras passou limpo. Vou rodar a validação completa do projeto; se passar, publico as regras no database nomeado, já que você autorizou configurar e incluir pelo Firebase CLI.

>

> Validação completa passou e o build já mostrou ganho real: as telas agora saem como arquivos separados por rota/área, em vez de tudo preso no bundle principal. Vou publicar as regras/indexes no Firestore nomeado agora.

>

> Regras e indexes foram publicados no database correto. Vou subir uma instância local limpa e rodar os smoke tests para garantir que o BFF ainda responde depois da mudança do Admin SDK para banco nomeado.

>

> Vou checar se a porta nova terminou de subir e, em seguida, rodar os fluxos.

>

> Servidor local saudável. O aviso é só a porta de hot reload do Vite já ocupada; não impede API nem página. Vou rodar os testes de fluxo.

>

> Como mexemos em carregamento visual das rotas, vou tentar uma checagem pelo navegador interno. Se o ambiente repetir a falha anterior do plugin, fico com as validações de build/HTTP que já passaram.

>

> MCP tool call

>

> node_repl.js

>

> ```json

> {

> "code": "const { setupBrowserRuntime } = await
import('file:///C:/Users/marco/.codex/plugins/cache/openai-
bundled/browser/26.609.30741/scripts/browser-client.mjs');\nawait
setupBrowserRuntime({ globals: globalThis });\nglobalThis.browser = await
agent.browsers.get('iab');\nnodeRepl.write(await browser.documentation());",

> "timeout_ms": 30000,

> "title": "Conectar navegador interno"

> }

> ```

>

> ```text

> failed to write kernel assets: O sistema não pode encontrar o caminho especificado. (os error 3)

> ```

>

> O navegador interno repetiu a falha do ambiente, então vou registrar isso como limitação e fechar a entrega com as validações automatizadas. Antes, vou atualizar os documentos da auditoria/plano com esta fase.

>

> <details><summary>Edited a file</summary>

>

```
> - Wrote `./Plataforma-  
Coroado/IMPLEMENTACOES_AUDITORIA_COROADO_2026-06-14.md` (+28 -0)  
  
>  
  
> </details>  
  
>  
  
> Vou também atualizar o plano de evolução para refletir que agora o Firestore real  
está conectado e que a migração por rota saiu do papel para chunks de tela.  
  
>  
  
> <details><summary>Edited a file</summary>  
  
>  
  
> - Wrote `./Plataforma-  
Coroado/PLANO_EVOLUCAO_STACK_AUDITORIA_COROADO_2026-06-14.md`  
(+18 -0)  
  
>  
  
> </details>  
  
>  
  
> Vou encerrar a instância local de teste e fazer uma última checagem de estado  
dos arquivos.  
  
</details>
```

Avancei a fase e já deixei o Firebase real configurado.

O principal: não criei um Firestore novo porque o projeto já tinha o database correto ativo. Configurei a plataforma para usar esse banco nomeado em todos os pontos relevantes:

```
`gen-lang-client-0529830528 / ai-studio-534c2e7e-8664-4b76-95e3-  
faf31fc1628b`.
```

Também publiquei as regras/indexes no Firestore real via Firebase CLI.

Mudanças principais:

- Criei [.firebaseerc](./Plataforma-Coroado/.firebaseerc)
- Ajustei [firebase.json](./Plataforma-Coroado/firebase.json) para o database nomeado
- Alinhei o BFF/Admin SDK em [src/server/context.ts](./Plataforma-Coroado/src/server/context.ts)
- Alinhei Cloud Functions em [functions/src/index.ts](./Plataforma-Coroado/functions/src/index.ts)
- Documentei operação Firebase em [docs/firebase/README.md](./Plataforma-Coroado/docs/firebase/README.md)
- Adicionei scripts `firebase:databases`, `firebase:indexes`, `firebase:rules:dry-run` e `firebase:rules:deploy`
- Avancei a migração das telas com carregamento sob demanda por rota em [src/App.tsx](./Plataforma-Coroado/src/App.tsx)

Sobre `DATABASE\_URL`: ainda não existe Cloud SQL/PostgreSQL para gerar essa URL. Então deixei `DATABASE\_URL` e `CLOUD\_SQL\_DATABASE\_URL` documentados como opcionais, mantendo o Firestore como banco operacional e o SQL/BI pronto para ligar quando a instância existir. Isso segue o modelo oficial de Firestore nomeado, em que o client precisa informar o databaseId ao instanciar o banco, e o Firebase CLI permite listar/gerenciar databases e publicar regras por database configurado no `firebase.json`: [Firestore databases](https://firebase.google.com/docs/firestore/manage-databases) e [deploy de rules por database](https://firebase.google.com/docs/rules/manage-deploy).

Validações executadas:

- `npm run firebase:databases -- --json`
- `npm run firebase:indexes -- --json`
- `npm run firebase:rules:dry-run`
- `npm run firebase:rules:deploy`
- `npm run validate:full`
- `FLOW\_BASE\_URL=http://localhost:4182 npm run test:flows`

Resultado importante do build: as telas agora saem em chunks separados, como `SchoolView`, `EventsView`, `FinanceView`, `StoreView`, `CellsView`, `AdminView`, etc. Isso já é a primeira migração real por rota, reduzindo o peso inicial e preparando a extração de páginas Next nativas. A única limitação: o navegador interno do Codex voltou a falhar por erro do ambiente do plugin, então a verificação ficou por build, Firebase CLI e smoke tests HTTP.

> Essa migração que estamos fazendo foi a melhor decisão técnica? O Next seria o que temos de mais moderno?

> Estamos caminçando para um estado de microservicos com fastAPI?

> Pode avançar pro proximo passo natural?

<details><summary>19 previous messages</summary>

> A resposta curta: sim, a direção está boa, mas com uma nuance importante. Vou explicar o critério técnico e já avançar no próximo passo natural: pegar uma área com BFF maduro e começar a transformá-la em página Next nativa de verdade, não só “SPA embrulhada”.

>

> <details><summary>Searched the web</summary>

>

> <details><summary>Searched the web</summary>

>

> - Searched the web for `Next.js official App Router Server Components React framework docs 2026`

>

> </details>

>

> </details>

>

> Tecnicamente: Next não é “a única coisa moderna”, mas para a Coroadó é a melhor ponte agora porque resolve interface, performance e rotas sem trocar tudo de uma vez. FastAPI faz sentido depois, como serviço especializado para IA, análise, recomendação e processamento assíncrono, não como substituto imediato do BFF Node/tRPC.

>

> Vou começar por Financeiro. É uma boa primeira tela nativa porque já tem regras, rotas backend, conciliação, indicadores e relação direta com o futuro SQL/BI. A meta desta passada é criar um “resumo financeiro” no BFF e uma primeira experiência Next para a rota, mantendo a tela completa atual como fallback operacional.

>

> Enquanto implemento, estou deixando a fronteira limpa: a consulta financeira vira uma função do servidor reaproveitável por REST, tRPC e Next. Isso evita criar uma página bonita que depois precise reescrever a regra de negócio.

>

> Para funcionar tanto no servidor Express atual quanto no Next App Router, vou criar a mesma rota nos dois mundos usando a mesma consulta interna. Essa é a ponte certa: não duplica lógica e já prepara a troca futura de servidor.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroadó/src/server/context.ts` (+4 -0)

>

> </details>

>

> Agora vou criar a consulta financeira compartilhada. Ela vai consolidar receita, pendências, recorrência, assinaturas, campanhas e últimas transações pelo `tenantId`.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/queries/financeOverview.ts` (+112 -0)

>

> </details>

>

> Com a consulta pronta, vou expor isso no REST atual e também no tRPC, para o mesmo dado alimentar a SPA e as páginas Next.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/financeRoutes.ts` (+16 -0)

>

> </details>

>

> Vou adicionar o mesmo resumo ao tRPC. Isso ajuda na migração porque telas Next futuras podem usar cliente tipado em vez de REST puro.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/trpc.ts` (+8 -0)

>

> </details>

>

> Agora vou criar a rota API dentro do próprio Next. Assim, quando rodarmos `next dev` ou App Router em produção, `/api/finance/overview` também existe sem depender do servidor Express.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/api/finance/overview/route.ts` (+23 -0)

>

> </details>

>

> Vou criar a primeira tela Next nativa do Financeiro com um painel de indicadores reais vindo do BFF. Ela não substitui ainda todos os formulários da tela antiga, mas já coloca a rota em Next com dados próprios e autenticação Firebase.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/financeiro/FinanceiroNativeClient.tsx`
(+244 -0)

>

> </details>

>

> Vou trocar a página `/financeiro` para carregar esse painel Next nativo em vez de apontar só para a SPA encapsulada.

>

> <details><summary>Edited 2 files</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/financeiro/FinanceiroNativeClient.tsx`
(+2 -2)

> - Wrote `./Plataforma-Coroado/src/app/financeiro/page.tsx` (+20 -2)

>

> </details>

</details>

> A imagem e os dados a seguir são sobre nosso banco de dados (criado):

> Cloud SQL

> Local: us-east1

> Instância: gen-lang-client-0529830528-instance

> Banco de dados: gen-lang-client-0529830528-database

>

> avance para conclusao.

>

> preciso que faça a sincronização com o github, e que deixe o build construido de forma que nao ira quebrar nossa plataforma que ja esta hospedada no google cloud, usando o firebase

>

> User context

>

> Images:

> - `C:/Users/marco/AppData/Local/Temp/codex-clipboard-ac7daa41-f482-42f8-b632-ca661ed85ab0.png`

<details><summary>1 previous message</summary>

> Recebi os dados do Cloud SQL/Data Connect. Vou fechar essa etapa com três frentes: ligar a configuração não sensível do Cloud SQL ao projeto, validar que a migração/build continuam estáveis, e depois sincronizar com o GitHub de forma rastreável.

</details>

Vou usar o fluxo de GitHub do ambiente para evitar publicar coisa pela metade. Primeiro confirmo branch/remotes e depois faço commit só após os testes passarem.

> continue as ultimas solicitacoes que ainda nao finalizamos

<details><summary>34 previous messages</summary>

> Vou retomar exatamente de onde ficou aberto: terminar a primeira tela Next nativa do Financeiro, ligar os metadados do Cloud SQL sem inventar senha/URL inexistente, validar tudo e então sincronizar no GitHub. Primeiro vou reabrir o estado do repositório e a orientação do fluxo GitHub.

>

> <details><summary>Searched the web</summary>

>

> <details><summary>Searched the web</summary>

>

> - Searched the web for `Next.js App Router Server Components official documentation`

>

> </details>

>

> </details>

>

> Agora vou ligar os dados do Cloud SQL que você enviou como configuração operacional segura. Como ainda não temos usuário/senha/URI, vou registrar instance/database/service e preparar o worker para aceitar Cloud SQL por socket/variável quando a credencial existir, sem quebrar o build atual.

>

> Vou ajustar o worker para aceitar tanto uma `DATABASE\_URL` completa quanto conexão Cloud SQL por metadados + credenciais. Assim já fica pronto para a instância criada, mas sem hardcode de senha no repositório.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/scripts/sync-firestore-to-sql.ts` (+44 -8)

>

> </details>

>

> Agora vou registrar os dados do Cloud SQL no exemplo de ambiente e na documentação do BI para ficar claro o que já temos e o que ainda precisa ser preenchido como segredo.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/.env.example` (+9 -0)

>

> </details>

>

> Vou adicionar um arquivo operacional específico para Cloud SQL, com os IDs que você enviou e o modo seguro de ativar a sincronização.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/docs/sql-bi/cloud-sql-coroado.md` (+38 -0)

>

> </details>

>

> Vou corrigir um detalhe de caminho na nova página financeira Next antes de validar.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/financeiro/FinanceiroNativeClient.tsx` (+1 -1)

>

> </details>

>

> Vou atualizar o README do BI para apontar para o Cloud SQL criado, em vez de deixar a seção como se ainda fosse totalmente futuro.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/docs/sql-bi/README.md` (+9 -1)

>

> </details>

>

> Vou rodar a tipagem agora. Se algo da nova rota Next ou do worker reclamar, corrijo antes de qualquer commit.

>

> Tipagem passou. Vou validar o worker com o Cloud SQL sem credenciais, para garantir que ele reconhece a instância criada e não tenta escrever sem segredo.

>

> O worker reconheceu a conexão Cloud SQL correta e parou de forma segura por falta de credenciais. Vou rodar a validação completa, incluindo Firebase rules, Vite e Next.

>

> A validação completa passou e o Next agora mostra `/financeiro` e `/api/finance/overview` como rotas dinâmicas reais. Vou incluir essa rota nos smoke tests para garantir que continue exigindo autenticação.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/scripts/flow-smoke-tests.mjs` (+1 -0)

>

> </details>

>

> Vou registrar também a decisão arquitetural: Next como shell principal, Node/tRPC como BFF e FastAPI apenas quando houver motivo claro de serviço especializado, especialmente IA/dados.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/docs/architecture/stack-decision.md` (+52 -0)

>

> </details>

>

> Vou atualizar o registro da auditoria com esta etapa de conclusão: Cloud SQL, rota financeira nativa e decisão de arquitetura.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/IMPLEMENTACOES\_AUDITORIA\_COROADO\_2026-06-14.md` (+23 -0)

>

> </details>

>

> Vou fazer a validação operacional final agora: build das Functions, dry-run do worker SQL, servidor local e smoke tests com a rota financeira nova.

>

> Essas três validações passaram. Vou subir uma instância local limpa e executar os testes de fluxo contra ela.

>

> Vou confirmar a porta e rodar os fluxos.

>

> Smoke tests passaram com a rota financeira protegida. Vou encerrar a instância local de teste antes de preparar o commit.

>

> Agora vou preparar a sincronização com GitHub: checar autenticação do GitHub CLI, criar uma branch de trabalho a partir da `main`, revisar o escopo e commitar a entrega validada.

>

> O GitHub CLI está instalado, mas a sessão `gh` está expirada. Vou mesmo assim criar commit local numa branch segura; depois tento o push via Git. Se o Git também pedir autenticação, deixo tudo pronto e explico exatamente o que falta para enviar ao GitHub.

>

> Branch criada. Vou stagear o conjunto inteiro da auditoria/migração, porque o escopo aqui é a entrega completa que vínhamos construindo: Firebase, Next, BFF, SQL/BI, Mercado Pago, motion, docs e validações.

>

> Arquivos stageados. Vou criar o commit local agora; depois tento enviar para o remoto.

>

> Commit local criado. Vou tentar enviar a branch para o GitHub remoto.

>

> A branch já subiu para o GitHub. Vou tentar abrir o PR; se a autenticação do `gh` impedir, deixo o link direto de criação que o próprio GitHub retornou.

>

> PR criado. Vou só confirmar o estado final da branch e fechar com o que foi sincronizado, o que já está seguro para Firebase/Google Cloud e o que ainda depende de segredo do Cloud SQL.

>

> Está tudo sincronizado: branch limpa, commit no remoto e PR draft aberto. vou consolidar o fechamento com link, validações e decisão técnica.

</details>

Sim: a decisão técnica está correta. Next.js é o melhor caminho agora para a Coroadó porque permite modernizar rota por rota sem quebrar a plataforma

hospedada. Ele é moderno no ponto certo: App Router, rotas por arquivo, APIs, Server Components e carregamento progressivo. A própria documentação do Next posiciona o App Router como o roteador mais novo, com suporte às features atuais de React e Server Components.

FastAPI também é moderno, mas eu não migraria o núcleo para FastAPI agora. O melhor caminho é: Next + Node/tRPC como BFF principal, e FastAPI depois para serviços especializados de IA, análise, recomendação, automações pesadas ou jobs assíncronos. A documentação do FastAPI o define como framework moderno e de alta performance para APIs Python, mas microserviço é uma decisão de arquitetura, não uma obrigação da ferramenta.

Avancei e concluí a etapa:

- Cloud SQL registrado:
- `us-east1`
- instância `gen-lang-client-0529830528-instance`
- banco `gen-lang-client-0529830528-database`
- serviço `gen-lang-client-0529830528-service`
- Worker `sync:bi` agora aceita Cloud SQL por `DATABASE\_URL`,
`CLOUD\_SQL\_DATABASE\_URL` ou socket/TCP com `CLOUD\_SQL\_USER` e
`CLOUD\_SQL\_PASSWORD`.
- Criei a primeira página Next nativa em `/financeiro`.
- Criei `/api/finance/overview` no Express/BFF, tRPC e App Router.
- Documentei a decisão de stack em [stack-decision.md](./Plataforma-Coroadoc/docs/architecture/stack-decision.md).
- Documentei Cloud SQL em [cloud-sql-coroadoc.md](./Plataforma-Coroadoc/docs/sql-bi/cloud-sql-coroadoc.md).

Validações concluídas:

- `npm run validate:full`
- `npm --prefix functions run build`

- `npm run sync:bi -- --dry-run`
- `npm run firebase:rules:dry-run`
- `FLOW\_BASE\_URL=http://localhost:4183 npm run test:flows`

Sincronização GitHub feita:

- Branch: `codex/coroado-next-firebase-sql`
- Commit: `d44b44a Migrate Coroado platform to Next BFF and BI foundation`
- PR draft: <https://github.com/Mphubner/Plataforma-Coroado-v2/pull/1>

Não fiz deploy de Hosting para não trocar a plataforma hospedada sem revisão final do PR. O build está validado para não quebrar a base atual Firebase/Google Cloud; o próximo passo seguro é revisar o PR e então fazer deploy controlado.

Fontes usadas: [Next.js Docs](<https://nextjs.org/docs>), [Next.js App Router](<https://nextjs.org/docs/app>), [FastAPI Docs](<https://fastapi.tiangolo.com/>).

> proximos passo natural? da pra avançarmos?

<details><summary>46 previous messages</summary>

> Dá para avançar sim. O próximo passo natural é migrar a segunda área para Next nativo. Eu escolheria `Eventos`, porque já tem BFF, pagamento Mercado Pago, check-in e impacto direto na operação. Vou fazer uma versão Next nativa de leitura/visão geral, mantendo a tela completa antiga funcionando no build Firebase atual.

>

> Vou começar conferindo se a branch está limpa e se ainda estamos no PR certo antes de mexer. A ideia é somar um commit incremental ao PR já aberto, sem alterar a plataforma hospedada ainda.

>

> Context compaction

>

> Source: automatic

> Status: completed

>

> Sim, dá para avançar. O próximo passo natural é tirar mais uma tela crítica do modo “casca antiga dentro do Next” e levar Eventos para uma página nativa, porque ela cruza inscrições, pagamento, check-in e indicadores operacionais.

>

> A branch está limpa e o PR continua aberto como rascunho. Vou fazer uma entrega incremental bem delimitada: servidor de leitura de Eventos, rotas de API, tela nativa e validação, sem mexer na hospedagem atual do Firebase.

>

> Achei um padrão bom já criado em Financeiro: uma consulta central no servidor, uma rota BFF e uma tela Next enxuta. Vou reaproveitar esse desenho para Eventos para manter a migração consistente.

>

> Antes de editar, estou conferindo os contratos compartilhados para não criar uma “segunda verdade” para Eventos. A ideia é fazer a tela nova consumir os mesmos nomes de coleções e manter o caminho antigo intacto até a migração rota a rota acabar.

>

> Vou editar agora. A mudança principal é criar um resumo de Eventos que já nasce com indicadores: próximos eventos, vagas, inscrições, check-ins e pendências de pagamento.

>

> Também vou deixar a API de Eventos funcionando em dois modos: público para listar próximos eventos e autenticado para trazer indicadores de gestão/check-in quando o usuário tiver permissão. Isso preserva a experiência aberta sem expor dados operacionais desnecessários.

>

> Começando pelas peças de backend. Isso dá base para a tela e para futuros relatórios, porque os indicadores passam a ser calculados num ponto central em vez de espalhados pela interface.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/context.ts` (+15 -0)

>

> </details>

>

> O resumo de Eventos vai aceitar dados legados sem `tenantId` apenas como pertencentes ao tenant padrao. Assim a migracao nao some com eventos antigos da tela, mas ainda preserva o caminho multiunidade para dados novos.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/queries/eventsOverview.ts` (+272 -0)

>

> </details>

>

> Agora vou expor esse resumo pelo BFF existente e pelo App Router do Next. Com isso a mesma regra serve tanto para o servidor Express atual quanto para a nova camada Next.

>

> <details><summary>Edited 2 files</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/eventRoutes.ts` (+13 -0)

> - Wrote `./Plataforma-Coroado/src/server/trpc.ts` (+8 -0)

>

> </details>

>

> A API pública/autenticada de Eventos está entrando em `/api/events/overview`. A seguir vou conectar a rota equivalente do Next, que é a peça usada pela página nativa nova.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/api/events/overview/route.ts` (+18 -0)

>

> </details>

>

> Pequeno ajuste de caminho interno na rota Next: ela precisa apontar para `src/server`, igual a rota financeira nativa.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/api/events/overview/route.ts` (+2 -2)

>

> </details>

>

> A parte visual nativa vai entrar com uma tela operacional, não apenas uma lista. Ela vai mostrar o que está por vir e, quando houver permissão, também gargalos de pagamento/check-in.

>

> Vou aproveitar para tornar a inscrição de evento reutilizável também. Assim o Express, o tRPC e uma futura rota Next de inscrição podem chamar a mesma operação, em vez de duplicar regra de pagamento.

>

> Essa extração melhora a robustez: se amanhã Eventos estiver servindo por Next, Express ou tRPC, a regra de capacidade, pagamento e Mercado Pago continua em um único lugar.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/operations.ts` (+132 -1)

>

> </details>

>

> Vou reduzir a rota Express de Eventos agora: ela deixa de carregar a regra inteira e passa a chamar a operacao compartilhada. Isso diminui risco de divergencia entre a tela antiga e a nova.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/eventRoutes.ts` (+6 -129)

> - Wrote `./Plataforma-Coroado/src/server/routes/eventRoutes.ts` (+1 -1)

>

> </details>

>

> Com a operacao compartilhada pronta, vou criar tambem a rota Next de inscricao. Isso evita uma tela nativa que consiga listar eventos, mas dependa da API antiga para a acao principal.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/api/events/[eventId]/enroll/route.ts` (+37 -0)

>

> </details>

>

> Vou manter a tela com acoes simples: atualizar, entrar e inscrever. A inscricao sem pagamento continua criando o ingresso direto; evento pago retorna o link do Mercado Pago e redireciona.

>

> Agora vai a parte da tela em si. Estou mantendo o visual mais operacional e compacto, sem transformar a pagina em landing page, porque Eventos precisa ser usado repetidamente por equipe e membros.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/eventos/EventosNativeClient.tsx` (+363 -0)

>

> </details>

>

> A tela nativa foi criada. Vou trocar o wrapper antigo da rota `/eventos` por essa nova pagina Next, mantendo a SPA antiga ainda disponivel no bundle legado para as outras rotas.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/eventos/page.tsx` (+20 -2)

>

> </details>

>

> Vou fazer uma limpeza rapida nos textos novos para manter o padrao ASCII do repositorio nesta entrega e evitar problemas de encoding no build/documentacao.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/app/eventos/EventosNativeClient.tsx` (+1 -1)

>

> </details>

>

> A base da tela já está conectada. Antes de validar, vou registrar essa entrega nos documentos da auditoria para ficar claro o que entrou agora e o que fica como próximo lote.

>

> Chegou a hora de validar. Vou começar pelo TypeScript, porque ele pega rápido erros de contrato/caminho antes de eu gastar tempo com build completo.

>

> TypeScript passou limpo. Vou rodar a validação completa agora: regras Firestore com Java/emulador, build Vite e build Next.

>

> A validação completa caiu por falta de memória do Node logo no início, não por erro de código. Vou repetir com limite de memória maior para separar problema de ambiente de problema real da plataforma.

>

> Mesmo com limite maior, o comando agregado repetiu a queda de memória no mesmo ponto. Como o `lint` isolado passou, vou validar por etapas para identificar se é só o agregador/processo do Windows ou se algum build real quebrou.

>

> As regras passaram isoladamente. Há vários processos Node ativos na máquina, então vou continuar por comandos menores sem encerrar processos que possam ser do seu ambiente.

>

> Firestore validou com o Java portátil. Agora vou validar os dois builds separadamente.

>

> O build Vite chegou a transformar os módulos, mas o processo caiu por memória no empacotamento. Vou inspecionar quais processos Node estão ativos para ver se há servidores antigos prendendo memória antes de repetir.

>

> Vejo processos de suporte do proprio ambiente e alguns processos de ferramentas Firebase/Google que podem ter ficado de validacoes anteriores. Para nao derrubar nada que seja do Codex, vou tentar primeiro um build com memoria explicita e sem reaproveitar o script agregado.

>

> O build Vite passou com Node iniciado com memoria maior. Para que isso nao dependa de eu lembrar o comando manual, vou ajustar os scripts do projeto para usarem esse limite de memoria de forma padrao em lint/build/Next build.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/package.json` (+3 -3)

>

> </details>

>

> Scripts ajustados. Vou repetir a validação completa pelo caminho oficial do projeto; se passar, fica mais confiável para Cloud/Firebase e para qualquer pessoa rodando localmente.

</details>

Validação completa passou agora, incluindo Next com as novas rotas `/eventos`, `/api/events/overview` e `/api/events/[eventId]/enroll`. Vou subir o servidor local em porta isolada só para rodar os testes de fluxo HTTP e encerrar em seguida.

> sobre o database, o google cloud gerencia de forma diferente aparentemente, nossa maquina o gc cli ja esta logado no projeot inclusive, aparentemente nao vai ter de fato uma URL do database

>

> <https://docs.cloud.google.com/sql/docs/mysql/cloud-sdk?hl=pt-br>

>

> peço agora que faça uma entrega mais robusta que contemple a maior parte que conseguir para avançarmos com a plataforma, ao acessa-la ainda vi muitos dados estaticos (mockados) muitos dados que deveria ser de,

>

> preciso que os sistemas da pagina "Células", Escola Ide e Eventos, esteja 100% funcional

<details><summary>66 previous messages</summary>

> Entendi. Vou tratar isso em duas frentes: primeiro adaptar a visão do banco para o jeito real do Cloud SQL, sem depender de uma “URL mágica”; em paralelo vou atacar os dados mockados das três áreas críticas, começando por mapear exatamente o que ainda está estático em Células, Escola IDE e Eventos.

>

> <details><summary>Searched the web</summary>

>

> <details><summary>Searched the web</summary>

>

> - Searched the web for `https://docs.cloud.google.com/sql/docs/mysql/cloud-sdk?hl=pt-br`

>

> </details>

>

> </details>

>

> Pelo documento oficial, a CLI `gcloud sql` gerencia instancias via API Cloud SQL Admin e o fluxo normal é listar/configurar projeto e operar a instancia por comandos, não necessariamente por uma `DATABASE\_URL` única. Vou confirmar no projeto qual engine foi criada, porque o link que você mandou é de MySQL e nosso worker anterior estava preparado principalmente para PostgreSQL.

>

> Confirmei: a instancia real é PostgreSQL 18, não MySQL. Então nossa direção de SQL/BI com `pg` estava correta; o ajuste agora é documentar/automatizar Cloud SQL por `gcloud` /Auth Proxy/conector, e não esperar uma URL pronta no console. vou seguir para os dados reais das três áreas.

>

> O mapa inicial mostra que Escola ainda tem bastante texto/data estática, Células ainda tem metas/alertas/engajamento IDE fixos, e Eventos tinha fallback de data/local/imagem. Vou substituir o máximo viável agora por leituras reais e criar rotas de backend para as ações críticas.

>

> A estratégia aqui é pragmática: não vou “embelezar mock”; vou conectar os blocos que estavam fixos aos documentos que já existem no Firestore: membros, relatórios de célula, matrículas/progresso da Escola, cursos/aulas/eventos/ingressos. Isso já cobre a maior parte do que você viu como dado estático.

>

> Vou adicionar uma pequena camada de leitura BFF para a Escola e Células em vez de puxar coleções sensíveis direto no cliente. Isso ajuda a deixar os painéis funcionais e também prepara indicadores sem depender de permissões Firestore amplas no navegador.

>

> Vou mexer primeiro em Células porque ali havia dados fixos bem visíveis: 85% IDE, alertas com nomes fictícios, escalas locais e seletor multi-célula fictício. Vou conectar isso a membros, relatórios e matrículas reais.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/server/queries/cellOverview.ts` (+223 -0)

>

> </details>

>

> O resumo de Células agora calcula assiduidade, engajamento IDE, risco pastoral e escala sugerida a partir de dados reais. Vou expor isso pela API do backend e registrar no servidor.

>

> <details><summary>Created a file, edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/cellRoutes.ts` (+24 -0)

> - Wrote `./Plataforma-Coroado/server.ts` (+2 -0)

>

> </details>

>

> Vou adicionar um helper de leitura HTTP no frontend para essas consultas. Hoje só havia helper para POST, então várias telas acabavam lendo direto ou improvisando.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/lib/api/http.ts` (+28 -0)

>

> </details>

>

> Agora vou conectar o painel da célula a esse resumo, substituindo blocos estáticos por números e listas reais.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+43 -1)

>

> </details>

>

> O arquivo tem alguns trechos com encoding antigo, então vou aplicar as alterações em blocos menores para não correr risco de mexer em áreas erradas.

>

> Algumas linhas com acentos estão vindo diferente do que a ferramenta de patch espera. Vou trocar os trechos sem depender dessas palavras acentuadas.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/components/CellManagementDashboard.tsx` (+2 -2)

>

> </details>

>

> O principal indicador de multiplicação já deixou de ser fixo. Vou seguir nos demais números.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroador/src/components/CellManagementDashboard.tsx` (+2 -2)

>

> </details>

>

> O bloco de metas agora reflete dados reais da Escola IDE. Vou trocar o medidor fixo de 85% por engajamento calculado.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-

Coroado/src/components/CellManagementDashboard.tsx` (+3 -3)

>

> </details>

>

> Vou remover os nomes fictícios dos alertas pastorais. A partir daqui eles aparecem só quando o backend encontra risco real pelos relatórios/progresso.

>

> Agora encontrei o texto com a codificação correta. Vou aplicar o bloco novamente.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-

Coroado/src/components/CellManagementDashboard.tsx` (+11 -12)

>

> </details>

>

> Vou trocar também o “Meu engajamento” fixo de 100% por assiduidade e progresso IDE do usuário logado.

>

> Mesmo ajuste, agora com o texto real do arquivo.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-

Coroado/src/components/CellManagementDashboard.tsx` (+15 -3)

>

> </details>

>

> Na lista de membros também havia “IDE 100%” fixo. Vou transformar em status real por membro, vindo do resumo da célula.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/queries/cellOverview.ts` (+15 -0)

>

> </details>

>

> Agora vou refletir esse status real na lista de membros e conectar as escalas sugeridas.

>

> Vou fazer em partes, começando pelo tipo e pela assinatura da aba de membros.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+9 -1)

>

> </details>

>

> Vou trocar o `map` para calcular status individual antes de renderizar cada cartão.

>

> Mais um trecho com acento real. Vou aplicar com o texto correto.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+9 -4)

>

> </details>

>

> As escalas deixam de nascer com nomes fixos e passam a vir do backend. Se não houver escala persistida, o backend sugere distribuição com os membros reais da célula.

>

> Vou aplicar no texto original com acentos preservados.

>

> Vou substituir o bloco inicial da escala por um bloco alimentado pelo overview.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+18 -10)

>

> </details>

>

> A escala agora nasce de dados reais/sugestões reais. Vou remover a mensagem de “persistência futura” desse botão.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/CellManagementDashboard.tsx` (+1 -1)

>

> </details>

>

> Vou fazer o painel buscar o resumo do backend sempre que abrir uma célula, e passar esse pacote para as abas.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-

Coroado/src/components/CellManagementDashboard.tsx` (+21 -0)

>

> </details>

>

> Vou ligar as abas ao novo `overview`.

>

> Também vou remover o seletor fictício de “Célula Vida/Sal”; enquanto não houver uma tela de troca de célula real, ele deve mostrar apenas a célula atual.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-

Coroado/src/components/CellManagementDashboard.tsx` (+5 -8)

>

> </details>

>

> Células já ficou bem mais realista. Agora vou atacar Escola IDE, onde os painéis administrativos tinham números, gráficos, alunos, certificados e financeiro mockados. Vou criar um overview de backend para alimentar esses blocos.

>

> <details><summary>Created a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/queries/schoolOverview.ts` (+252 -0)

>

> </details>

>

> O overview da Escola calcula alunos ativos, cursos, conclusão, receita, certificados, alunos por ministério e alertas. Vou expor isso no BFF.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/server/routes/schoolRoutes.ts` (+16 -0)

>

> </details>

>

> Vou conectar o frontend da Escola ao novo overview. Primeiro a página principal, depois os painéis administrativos/certificados/conquistas.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+52 -1)

>

> </details>

>

> Agora vou carregar esse pacote quando o usuário autenticar.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+21 -11)

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+2 -0)

>

> </details>

>

> Vou substituir o progresso fixo do topo e passar o overview para as abas que precisam dele.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+5 -5)

>

> </details>

>

> Vou fazer Certificados e Conquistas refletirem o progresso real do aluno.

>

> A mesma questão de acentos apareceu na Escola. Vou trocar função por função usando o texto exatamente como está no arquivo.

>

> Vou substituir esses dois blocos inteiros.

>

> Para não perder tempo brigando com encoding antigo, vou fazer de forma segura: criar componentes novos alimentados por dados reais e trocar a renderização para eles. Os blocos antigos ficam sem uso e depois fazemos uma limpeza mais calma.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+2 -2)

>

> </details>

>

> Vou adicionar os novos componentes reais antes da área antiga.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+111 -4)

>

> </details>

>

> Agora o painel administrativo da Escola precisa usar os mesmos dados reais. Vou parametrizar o componente e trocar os principais blocos visíveis: métricas, gráficos, financeiro, membros e certificados.

>

> <details><summary>Edited a file</summary>

>

> - Wrote `./Plataforma-Coroado/src/components/SchoolView.tsx` (+5 -3)

>

> </details>

</details>

Vou substituir os cards e gráficos principais do admin por dados do overview.

> em seguida preciso que as telas dos membros, ministerios, social, gestão e pastores estejam 100% funcionais tbm